



Evaluation of Optimization Algorithm for Application Placement Problem in Fog Computing: A Systematic Review

Ankur Goswami¹ · Kirit Modi¹ · Chirag Patel²

Received: 17 October 2024 / Accepted: 24 January 2025 / Published online: 20 February 2025

© The Author(s) under exclusive licence to International Center for Numerical Methods in Engineering (CIMNE) 2025

Abstract

Application placement in fog computing is a crucial aspect of designing and managing fog computing systems. As a consequence, optimized application management becomes an essential task. Numerous algorithms have been proposed in the literature for efficient placement of applications in fog to address the Fog Application Placement Problem (FAPP), however not much work was done in evaluation of optimization algorithms. The evaluations are needed for making practical and impactful decision that can contribute to the scalability, reliability, and cost-effectiveness of the fog. This evaluation work is targeted towards the search of most efficient algorithms for FAPP especially optimization algorithms. The evaluations presented in this work provide answers to the most essential research questions such as what is the need of optimization in fog environment, which optimization algorithm performs best and what is its future in fog computing. The evaluations in this paper firstly, focuses on metric-based evaluations, which evaluates Fog Utilization, Service Level Agreement (SLA) violation and Response time of various state of the art works collected from the literature. The work also evaluates different type of Optimization algorithms and Optimization objectives side-by-side to see which type are best suited for FAPP.

1 Introduction

The Internet of Things (IoT) era was shaped by the technical improvements in processors, memory, and communications, which enabled the incorporation of computing power into a variety of objects [27]. As a result of this drift, massive data-the “*Big Data*” are collected by IoT sensors and is kept at cloud Data Centres (DC) [42]. In terms of delivering almost infinite processing and storage capacity that is available from anywhere, cloud computing is undoubtedly highly successful. Although data processing rates have improved dramatically, the bandwidth required to transport data to and

from data centres has not increased significantly [22], which may result in long latency.

The number and geographic spread of IoT devices make it more challenging to facilitate the movement of data from and to, to thousands of such devices. Also, it is obvious that there is a need to provide processing or storage closer to where data is created around the clock, to minimise necessary connectivity needs, and to latency for time-sensitive applications. According to [103], cloud computing provides virtualized resources built upon a shared infrastructure that consumers may utilise on a pay-per-use basis with a range of pricing options. It is built on numerous data centres that are dispersed across the globe and are positioned in the network’s centre and accessible via the Internet. But it is not close to the user where data is being created hence consumes more bandwidth and results in latency.

The improved computing capacity of mobiles phones as well their desired integration with new smart devices that integrate with variety of devices working on the same application or service, ask for various individual resource requirements. Due to its aforementioned features, these novel computational needs, which are defined by the requirement for a local processing paradigm, are not well addressed by cloud computing [30]. As a counterbalance to

✉ Ankur Goswami
ajgoswami.ce@gmail.com

Kirit Modi
kiritmodi@gmail.com

Chirag Patel
chiragpatel.dce@charusat.ac.in

¹ Faculty of Engineering and Technology, Sankalchand Patel University, Visnagar, Gujarat 384315, India

² Department of Computer Engineering, Charotar University of Science and Technology, Changa, Gujarat 388421, India

the vulnerability of cloud computing, FC has emerged as a potential solution to the growing demand to boost processing, network, and storage capacity nearer to end users.

2 Fog Computing (FC)

A novel virtual computational model named Fog Computing brings typical cloud computing and services out to the network's edge i.e., near end users. It integrates cloud centre and network edge devices very smoothly. It [45] provides processing, communication, computation, storage and services capabilities at the edge of network by using resources locally at edge. FC is a geographically dispersed computing where diverse devices at the network's edge are constantly linked to one another to cooperatively provide elastic compute, communication, and storage services. Cisco originally developed fog computing so as to extend cloud computing to a network's edge. The word 'fog' is simply used because "fog is a cloud close to ground" [21]. Since it operates at the network's edge, as a result, FC can decrease the time taken by application services to be delivered and avoid network congestion in contrast to cloud.

By combining local resources, FC provides control, processing, communication, and storage closer to the user. The widely spread network edge devices consume data. Consequently, the volume of network transmission and data transfer time are considerably minimised. Real-time or latency-sensitive application requirements may be successfully met by the fog paradigm, which also significantly reduces network bandwidth constraints. FC structure places an extra layer which is rich in resources between user and the cloud so as to tackle problems with low latency, high performance, high reliability and security, and interoperability. There are numerous fog nodes in the fog platform. These fog nodes have different network edge devices and management systems within devices, and also include some virtualized edge DC. Hence precisely, instead serving as a replacement for cloud computing, FC is an extension and augmentation of cloud. These data produced by sensors and user devices are processed and stored by fog nodes. Then, if the useful data that is still left is sent to a cloud server for storage or further computation.

2.1 Traditional Cloud Computing and Its Limitations

Traditionally, most computation take place in the cloud, which is a centralised computing paradigm. This indicates that a single, central cloud must receive all requests and data. Certainly, the speed for data processing have been amplified, but the network capacity has not significantly improved. Because there is so much data, the network capacity is now

the cloud computing industry's bottleneck [13]. Long delay might be the outcome. Data transport delays must not be tolerated. Additionally, certain choices can be made locally rather than being sent to the cloud.

To overcome these issues, cloudlet [45] leverage computing resources close to users to accomplish local processing and storage while minimising network transmission and latency [25]. Cloudlets is accessed only through which results in low coverage area, consequently, it cannot provide ubiquitous computing. Mobile Cloud Computing (MCC) has also been suggested as a method to offer new service models for mobile consumers and fully use cloud computing [3]. It describes an architecture where some processing and analytical functions are performed on edge devices, and coordination and data archiving are handled by the cloud. But often these devices are constrained due to their battery or storage capacity.

The processing of enormous amounts of data, traffic congestion, high connection costs, and major end-to-end delays are only a few of the significant problems that cloud computing faces. Certain of these problems are primarily brought on by the great physical distance between the data centres (DCs) of cloud service providers, like Google, Apple, Amazon Web Services (AWS), Facebook, Anaconda web, Matrix, Microsoft, TATA, China Telecom, China Unicom, etc. As per the Natural Resources Defence Council (NRDC), in 2013, U.S [74], these DCs work 24/7 causing carbon footprints. In total, DCs consumed 91 billion kilowatt-hours (kWh) of electricity, and by 2020, it is anticipated that this number would rise to over 140 billion kWh, or the output of 50 power plants yearly. Due to this, US firms will incur yearly power costs of \$13 billion and produce around 100 million metric tonnes of carbon emissions. Most cloud service providers have geographically dispersed DCs, which results in significant DC to end user delays and Quality-of-Service (QoS) deterioration, making them unsuitable for time-sensitive service requests [29].

3 Fog Features and Characteristics

As per National Institute of Standards and Technology (NIST), the essential features of FC [45, 84, 40] can be summarized as:

Low Latency When compared to activities carried out by a cloud service, FC nodes are nearer to the end users and can provide a quicker analysis along with reaction to the data created and requested by the users. It could take longer to obtain a response from the server, much like with cloud services [4]. While computing the information of edge devices, it reduces waiting time. Moreover, it helps with

position responsiveness by allowing fog nodes to be hosted everywhere.

Geographic Distribution applications as well services running on a FC model necessitate geo-distributed placement [10] and administration, unlike cloud computing. For a quicker response, fog nodes are positioned close to the user and are geographically dispersed. In contrast to the integrated cloud, FC offers decentralised services and applications that may be hosted anywhere.

Heterogeneity Information gathered from many sources and processed using a variety of network connection methods is possible with FC. In fog computing resources, services, applications interoperate with each other [28]. As they are created by different firms and have different origins, edge devices or fog nodes require hosting according to where they will be used. Therefore, Fog can adapt to different systems.

Real-Time Interactions Fog computing applications and services go beyond batch processing to encompass real-time interaction. So as to deliver a real-time and quicker response, it processes services in real time while keeping the processing near to the user. In comparison to the batch analysis operated in the cloud, FC provide simultaneous communications among different fog nodes [61].

Scalability Fog computing supports resource flexibility by quickly detecting changes in network and device conditions as well as variations in workload response times. Hence supports scalability by operating various nodes at a time.

3.1 Fog Computing Architecture

Fog Computing provides processing, storage, and service network in a shared manner within Cloud Computing DC and end devices [84, 40]. FC's primary goal is to offer low latency for time-sensitive IoT operations [89]. Several researchers have created numerous reference architectures for fog. Fog computing environment's communication infrastructure can be built on either the Radio Access Network (RAN) or the Software Defined Network (SDN) [27]. The representation that is utilised the most is layered architecture. A fog computing infrastructure is frequently represented as a three-layered architecture.

Although the number of layers in the various designs described in literature varies, the IoT and Cloud layers are present in each and every one of them. As a result, the variants may be thought of as several ways to structure the Fog Computing. So, irrespective of the count of layers, the FC structure should be briefed in three crucial layers: IoT, Fog

and Cloud. Various IoT devices connecting at the network edge are represented by the IoT Layer. The end users of these gadgets can control them or they can autonomously gather environmental data. They will analyse and store the data they send in the Fog and Cloud levels. Devices known as Fog Nodes make up the Fog Layer consists of devices named as fog nodes [27]. These are "smart" devices that can process, store, and direct data to the Cloud Layer when necessary. Services that don't require a lot of processing power can be handled straight via fog layer, conserving cloud resources and reducing the possibility of traffic on cloud channels Fig. 1.

Also, in case of insufficient resources on the Fog Layer, before moving the data to the cloud, fog layer itself can serve as a mid-platform between the IoT and Cloud, delivering the necessary computing capacity for processing applications and services, e.g., data filtering and aggregation. The requests made by the IoT Layer devices can be processed more quickly by the Cloud Layer thanks to its more potent resources, especially those that the Fog Layer was unable to fully address. So, in a FC environment, the Cloud Layer also act as an important element.

With the increasing use of data usage, storage and processing, the scope and benefits [65] fog computing are rising. Fog manages a midway layer [112] between end user devices and Cloud DCs by using the computing components of the edge network. These computing devices e.g., laptops, Raspberry PIs, gateways, nano-servers, and micro-DC's—are referred to as fog nodes in fog environments. In close proximity to data sources, the Fog nodes run a various IoT applications. As a result, FC reduces the latency in data propagation and prevents a huge volume of data from being sent to cloud DC's. FC reduces the latency in data transmission by preventing massive amounts of data from being sent to cloud DCs. As a result, the service latency of diverse applications minimizes. Since fog sits between user and cloud, thus all the fog network and fog nodes are closer to the user, it helps conserving network bandwidth by using very little of it. To access cloud network, require higher bandwidth for a long period of time. The extent of network congestion is diminished via FC. By using FC, providers may distribute the computational burden around the network edge which is certainly away from cloud DC's, thus avoiding any congestion. FC reduces operating expenses for service providers, conserves energy for cloud DC's, and enhances user quality of experience (QoE). Fog also offers strong location awareness, which makes it easier to communicate with mobile and other low-power end user devices.

This paper focuses especially on the optimization algorithms for FAP problem which arises due to complex challenges inherent in fog computing environments. Optimization algorithms can and do offer approaches for

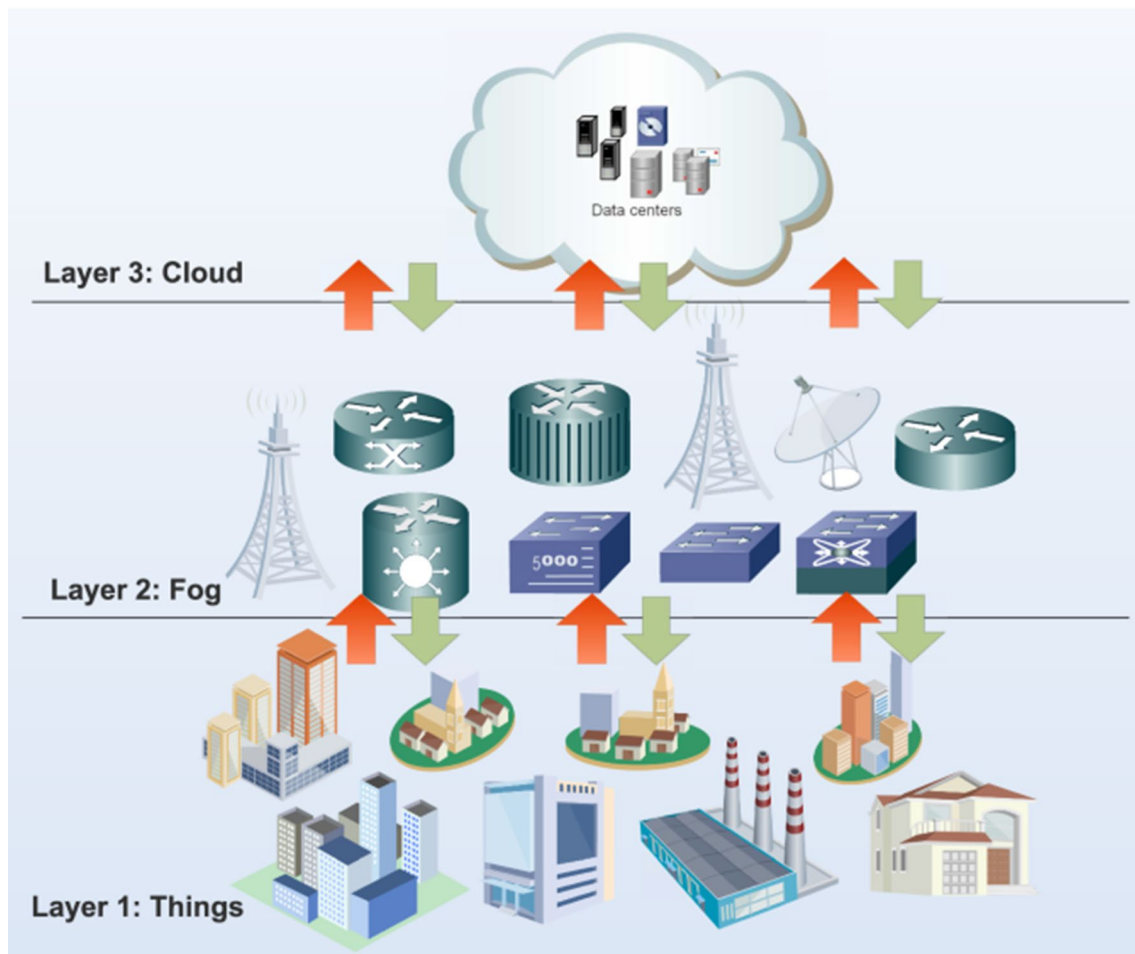


Fig. 1 Layers in Fog Architecture [91]

balancing competing objectives such as reducing response time, minimizing SLA violations, and maximizing resource utilization etc. However, there is a need of a study capable of systematically analysing and evaluating optimization algorithms. The Study not only can facilitate improved decision-making for newer deployments but also can reveal opportunities unexplored for developing novel optimization strategies that can address current limitations.

4 Motivation

FC is a distributed computing model which is an extension to the Cloud technology and is known to improve limitations such as bandwidth consumption and latency. Although Fog can certainly minimize application placement time and avoid network congestion, still a well-organized distribution of IoT application tasks in Fog setting is tedious. Hence well-organized application placement is challenging. So, various research articles are present in literature that have

covered several application placements challenges in Fog structure.

In [90] authors presented the basic idea about cloud computing and fog computing paradigms, quoted differences between both. They also pointed out the research gaps, and have addressed some ongoing research issues and challenges in fog. In [38] authors have provided a detailed overview of the computing offloading problem and also classified the algorithms as well as techniques that have been suggested recently for the computing offloading problem. Moreover, they also presented a novel categorisation of the optimization strategies that have been proposed to address the service placement problem along with mentioning the open challenges faced in fog environment. In [76] surveyed AI methods to resolve Fog Application Placement Problem (FAPP). Authors thoroughly compare and analyse AI algorithms in the FAP based Setting to look for the advantages as well as the shortcomings of AI- techniques and have also discussed future directions.

To the best of our knowledge, there is no work that has focused on providing comprehensive research of optimization algorithms in the Fog application placement problem.

As per our knowledge, none of the research has focused intensively on comparing and analysing optimization algorithms in the FAPP based on the advantages and disadvantages of all these algorithms. Hence comprehensive research is required to deliver an in-depth knowledge of the FAP challenges and opportunities based on optimization techniques aspect which can be quite useful for future research.

5 Contribution of this SLR

This SLR is followed by defining the required research questions, mentioned in Table 1 and the major contributions of this manuscript can be summarized as:

1. It provides a clear description of FC, its architecture, benefits and explored challenges in fog environment.
2. Provide details of Application Placement in fog environment and explained the related challenges.
3. Presented a classification summary of surveyed research based on application placement problem, taxonomy and optimization strategies.

Table 1 Research questions and motivations

S. No	Research Questions	Motivation
RQ1	What are the various challenging areas in fog computing?	Studying various challenges can give researchers insight to address unresolved issues.
RQ2	What are the challenges and issues faced in fog computing application placement?	The current status will help us to put light on what work has been done until now and will provide us a seed point to start work from.
RQ3	What is Application placement problem in Fog FAPP and what are various SOTA algorithms available for FAPP.	This will help identify various type of algorithms present in literature, which type of algorithms are lacking in the literature.
RQ4	What are the various Optimization algorithms present in the literature for fog computing application placement?	Learning different optimization algorithms will help authors to make the fog environment more efficient
RQ5	How to Compare and evaluate the optimisation algorithms and different types of optimization algorithms & optimisation objectives.	To find out the best type of optimization algorithm among all types of available optimizations and optimization objectives.
RQ6	What is the Future scope of application placement with optimization algorithms?	To discuss what areas, challenges have been researched and what still needs to be covered

4. The work focuses on two major evaluations for FAPP, first being the metric-based, which evaluates Fog Utilization, SLA violation and Response time of various SOTA works collected from the literature. The second evaluates different type of Optimization algorithms and Optimization objectives Side by Side to see which type of Algorithms are best suited for FAPP.
5. Finally, discussed few challenges and highlight the future scope of optimization algorithm in FAP.

6 Research Methodology

This SLR is about fog computing framework and the review course followed to perform this research involves: (i) Planning the review process (ii) framing research questions, (iii) stating rules for review, (iv) Data Collection, (v) Listing Search Criteria and Shortlisting and (vi) discussing future scope.

These RQs address key areas of concern in the FAPP, the findings can explicitly bridge existing gaps in optimization techniques by identifying and addressing specific shortcomings in the current methods. For instance, RQ1 and RQ2 will help identify major challenges in FAPP related to limitations in existing optimization algorithms. RQ3 and RQ4 will provide a detailed categorization of optimization algorithms and their ability to address these challenges in terms of scalability, adaptability, and computational efficiency. RQ5 will identify not only the best-performing optimization algorithms but also the trade-offs they entail, making way for the hybrid approaches. Finally, RQ6 will outline a clear roadmap for future development, specifying which unresolved issues from the review could be targeted by utilizing most recent optimization techniques. This well-structured framing of the RQs ensures that the findings directly inform the design of more effective and context-aware optimization solutions for FAPP, bridging the gap between current limitations and future possibilities.

6.1 Planning Review Process

The rules of review are designed by framing the research questions as discussed in Table 1. Then, we further searched these designed questions in the various databases. We follow a review technique is to recognize and collect sufficient evidences for the target study. The review process has a prominent part in consideration and non-consideration of articles. We prepare points on the review process and this report has been given to the peer authors. This process has repeated for a certain span of time. At last, the review rules have been finalized after detailed discussion of every

iteration. An extensive search has been performed on various electronic databases.

6.2 Research Questions

The aim of this study is to perform a systematic review on optimization algorithms used during application placement in fog computing. This study will help the researchers to understand the current status in the area of fog computing, related challenges, application placement along with optimization algorithm and discussion on its promising future. The research questions are required for motivation as well as to plan the review process.

6.3 Research Data Collection

There is a need to search extensively on electronic sources for conducting a systematic literature review. We have chosen the following set of databases to improve the chances of finding most related research papers: PubMed, Elsevier (sciencedirect.com), Wiley(wiley.com), Springer (link.springer.com), ACM Digital Library (www.acm.org/dl) IEEE eXplore (ieeexplore.ieee.org) Semantic Scholar (www.semanticscholar.org) Additional Sources: Google Scholar, DOAJ etc. We have considered the following additional sources in this systematic review to find other important studies: Explored the various articles mentioned in the bibliography of primary studies. Technical Reports Edited Books and Text Books.

6.4 Search Criteria

Different search strings (mentioned below) have been used to extract research matter from various electronic resources. Various famous scientific digital libraries and resources are

used to fetch research papers. The research papers have been obtained from different e-libraries e.g. DOAJ, PubMed, IEEE Xplore, Elsevier, ACM, Springer, Taylor and Francis (T&F), InderScience, and Google Scholar. The “Search string criteria” and “Search keywords selection” play a vital function in fetching relevant articles from the literature. The keywords used to describe the search string were fog computing, application placement, service placement and optimization with the use of boolean AND and OR operators to combine the keywords. The number of publications year wise using specified strings are mentioned in Fig. 2. The specified strings used are mentioned below:

Intitle: (“Service placement” OR “Application placement”) AND Intitle: “Fog Computing” AND Optimization

Intitle: (“Service placement” OR “Application placement”) AND Intitle: “Fog Computing” AND Intext: Optimization

Intitle: “Service placement” Intitle: “Fog Computing” Intitle: Optimization

6.5 Inclusion Criteria

Since the Fog Computing was first coined in 2012, we have set the article inclusion year from 2012 to 2024. We can clearly see that very few papers were published in its initial years but during the year 2015 the research paced up. Table 2 shows the number of published papers (year wise) for the considered research strategy. The above-mentioned

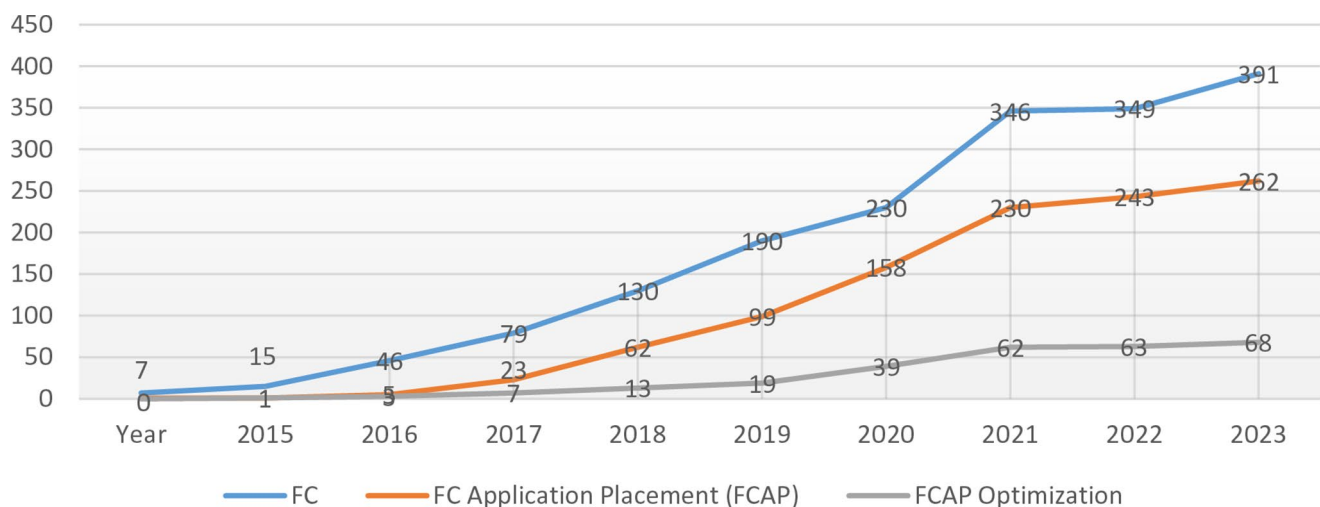


Fig. 2 Illustrating number of publications (Year wise) from selected e-resources using different search strings

Table 2 Number of publications (Year wise) from selected e-resources using the above-mentioned search strings

Year	FC	FC Application Placement (FCAP)	FCAP Optimization
2015	7	1	0
2016	15	1	1
2017	46	5	3
2018	79	23	7
2019	130	62	13
2020	190	99	19
2021	230	158	39
2022	346	230	62
2023	349	243	63
2024	391	262	68

search strings were used to fetch appropriate research articles from different digital libraries.

To identify quality papers, initially 200 papers both from various journals and conferences were selected. The quality article finding process has been performed to ignore articles that were not able to give any insightful or technical information and scientific discussion. Finally, 94 research papers were shortlisted from high-impact journals and conferences for this SLR. Most of the selected research papers are from IEEE and Springer comprising 21% and 18% of research matter (Fig. 3). Next sciencedirect 13%, ACM 5%, and rest researchgate, mdpi, pubmed, wiley with research matter 4% was included. Miscellaneous (27% of research matter was included) shows the clubbed research papers from the various journals from which only two or three research papers were considered for the study. So as to understand the basics, initially we have assessed research articles to

identify the research on FC at a high level. Further, detailed investigation was performed to select the most relevant research articles regarding application placement optimization. Excluding criteria includes:

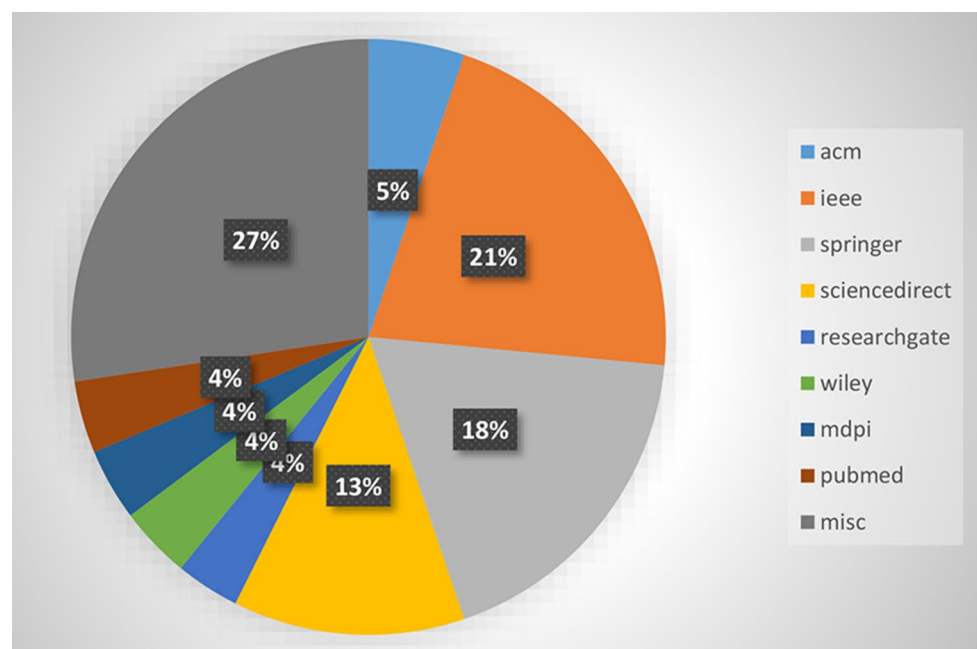
1. Short articles containing neither any insights nor any technical information or discussion.
2. The papers from low impact journals.
3. Non indexed journal and conference papers.
4. Articles which are not drafted in the English language.
5. The papers not following peer-reviewed process.

7 Related Surveys

Plenty of research articles are there that define fog FC [85] and discussed challenges related to FC. This SLR revied some of the articles for basic knowledge of FC [100, 112]. These research papers describe FC and it's related concepts very technically. Many articles [21, 22, 89] have been explored for learning the role and applications of Fog environment in various scenarios. Various research manuscripts have considered for better understanding of Fog Architecture [45, 84, 100103]. provides insight on emerging trends and technologies. Different papers discussed FC challenges in [73, 74, 84, 90]. Many research papers including surveys have put light on the crucial concept of application as well as service placement in FC details [22, 27, 39, 76, 85, 112].

These papers also discussed the challenges and difficulties faced regarding application placement in Fog structure. One of the challenges is Fog Application Placement Problem (FAPP), also addressed in various other recent research

Fig. 3 Most of the selected research papers are from IEEE and Springer comprising 21% and 18% of research matter respectively



papers [49, 92]. During the research we find out that there are many researchers have addressed various FAPP including Machine Learning (ML) and Deep Learning methods [11, 55]. Yet a survey of fog application placement problem with optimization algorithms is not explored deeply. So, This SLR aims to resolve the FAPP through optimization algorithms. Many research papers [18, 23] have been explored to study optimization algorithms to apply in Fog during application placement. This paper also discussed various performance metrics [110] to determine the efficiency of optimization algorithms.

8 RQ1: Research Challenges in Fog Computing

Many network, security, device, and IoT integration concerns exist in computing at the edge and are constantly being researched. The following are some problems that are taken into account by FC in a distributed environment:

8.1 Device and Network Challenges

Decentralized framework, networking resources, and device heterogeneity are some of the difficulties faced in relation to the device and network [84] and various other challenges are detailed below.

- **Decentralized framework:** The decentralised nature of the FC architecture results in a redundant structure. At the devices that are at the network's edge, the same code is repeated. The fog architecture thus concentrates on minimising redundancy in the decentralised design.
- **Networking resources:** The network resources are dispersed at random in the fog architecture. This makes connecting more challenging. It is possible to manage a suitable network that contains middle stage to keep a common collection of resources at the edge available for the needed application.
- **Device heterogeneity:** There are heterogeneous end devices in the fog environment. The structure is more variable because of the nature of heterogeneity [112]. This characteristic of heterogeneity at device and network should also be considered by the apps that are created employing fog.

8.2 Computational Challenges

Fog Computing faces numerous challenging aspects of computations at diverse and distributed levels which are detailed below.

- **Computations at different levels:** Cloud servers generally communicate with the fog system. The objective of the fog system is to reply to users at the lower level within a certain time frame and send the necessary calculations to the cloud, which will consume more time. When compared to other portions of the calculation, which will be done close to the edge with minimal computing costs, very few computations are offloaded to the clouds because they are not limited by reaction time or processing capacity. It can be challenging to determine which computations should be performed in the cloud and which should be in the fog.
- **Distribute computation resources:** The necessary resources might not be available for computation at the edge. Other fog nodes are a source of these materials. Due to this demand, it is now more important than ever to distribute resource computation among several fog nodes. This calls for a strategy that creates a common pool by combining the memory, computations, and networking resources. Applications from many fields can ask the pool and reserve resources as needed.
- **Computations in a Distributed Environment:** Distributed computation is used in the FC, where the accuracy of the calculations must be confirmed. The applications utilising the fog framework must be created in a way that minimises computation disparities (Li et al., 2017). These disparities should be provable.
- **Mobilization challenge:** In terms of mobilisation, OpenFog [68] is designated as an N-level structure. Yet, the sharp increase in the count of fog levels could result in long-term problems. As a result, level count must remain unwavering. Results of mobilisation will be approved if they are based on criteria such as the tasks performed at each level, the total number of sensors employed, the proficiency of fog devices, and latency of fog devices. Yet, it is crucial to consider how these requests will be complied.
- **Utilization of resource challenge:** Because the devices are varied in type and readily available, resource utilisation is at its most unique in an environment like a fog. Each of these fog devices is responsible for carrying out the application independently. For example, a computer that relies on administrative staff to handle some routine file operations and emails might be a part of the fog and this device can act as a fog device [44]. Using the resources that are available in this circumstance for computing the fog is active but expected via research on the long-term behaviour of its resources. This is crucial because, after the execution of the fog task has started and for a specific amount of time, the needs of the requested application, for which fog devices is responsible, may cause changes in the location of the resources.

- **Fog device breakdown challenge:** Because the fog devices are dispersed and their control is decentralised, the likelihood that they may malfunction is always increasing [40]. Thus, the fog devices may malfunction for a number of reasons, including end user behaviour, hardware failure, and software crashing. Apart from these problems, a few more that could be quite important include the power source, connection, and adaptability. Because that many fog devices may be connected by Wi-Fi, which is occasionally unreliable. As the bulk of them are wirelessly linked mobile devices, they could often switch locations to other clusters.
- **Request establishment handling challenge:** The fog manages thousands of intermittently responsive and non-responsive IoT devices and applications. In fog, the quality of service attainable and the environment are very different [57]. Hence, unique problem for the whole Fog land is making services accessible. In order to determine the possibility of fog-centred solutions, more study is required.
- **Complexity challenge:** Fog computing uses a variety of IoT sensors and devices that have been produced by several firms, making it challenging to choose the best mechanism due to varied hardware configurations, software setups, and individual needs [9, 84, 40]. Also, in a few cases, the requirements for maximum security application entail the use of specified protocols and devices, which increases the process's drawback.

8.3 Security Challenges

The heterogeneous devices that make up fog architecture are numerous. They could be exposed to various assaults, such as the man-in-the-middle attack in a foggy setting. Data and network are the two key difficulties that need to be resolved in the fog. The fog devices are established in a less secure range, making it easy to perform out any kind of attack. The devices at the edge must be function safely in the fog.

8.4 Service-Oriented Computing

Client services are divided into several little services and dispersed across cloud and edge in the fog computing architecture. A technique of service-oriented computation using edge equipment is this specialised dispersion of functions above the fog devices. Yet, providing modest services through fog nodes has its own set of problems [9, 58]. The main problem in the FC field is the proper structuring of the framework to acquire the services. The administration of small services is fraught with problems. The steps of implementation listed below include service combination, positioning, etc. In order for these services to be given to the

clients through the fog architecture in the shortest amount of time, we need a suitable composition structure.

8.5 Management of Resources

FC has the ability to modify and adapt to a variety of problems, including a lack of resources [36, 100] and temporary malfunctions. The entire system will become unresponsive if any fog node fails, and resources from that fog node won't be available. The fog environment uses virtualized versions of these resources. Resource virtualization presents a number of difficulties that must be effectively managed in order for the resources to remain accessible during downtime, including migration, latency, etc.

8.6 Fog Nodes and Cloud Integration

The supply of services or applications from the fog nodes to the cloud, together with QoS characteristics for various users, is another important consideration. The cloud and the edge devices make up the fog framework. To conduct compute and storage in a distributed structure, it is important to take care of the heterogeneous edge device orchestration as well as the cloud architecture. As a result, integration of between fog devices and the cloud servers must be allocated dynamically.

8.7 Emerging Paradigms

Fog computing and edge computing both extend cloud computing by bringing computation closer to the source of data, but they differ in their architectural scope and proximity to data sources. Fog computing operates as an intermediate layer between the edge devices and the cloud, facilitating the processing of data from multiple edge devices over a localized network. This network layer can encompass anything from local area networks to devices a few hops away from the edge. In contrast, edge computing typically refers to processing done directly on the devices themselves or on a local server situated physically closer to the devices, minimizing latency more significantly than fog computing by limiting data travel to the very edge of the network [38, 112]. This makes edge computing ideal for real-time applications requiring immediate processing, whereas fog computing offers a more scalable solution for handling data from larger networks of devices. In fog computing, optimization algorithms often need to focus more on real-time processing capabilities and by broadening the research on optimization algorithms will provide a more comprehensive view of the state-of-the-art in distributed computing paradigms.

9 RQ2: Application Placement in FC

FC provides a broad range of platform as well as infrastructure [65] services. Infrastructure services include the use of computing, networking (bandwidth and firewalls), and storage resources on demand, whereas platform services support application runtime environments, operating systems, and programming interfaces. Fog resource management refers to administrative tasks e.g., deployment, virtualization, and monitoring of Fog nodes that support infrastructure and platform services based on the Fog [95]. Furthermore, dynamic provisioning, load balancing and auto-scaling are implemented using Fog resource management to assure service availability and multi-tenancy. Application Placement is the assignment of application service components to infrastructure resources. The objective is to locate, traverse, and ideally choose among potential locations [39]. The FC architecture enables the use of resources in close proximity to IoT devices to provision latency-sensitive IoT applications of different sorts. Nonetheless, owing to the mobility of users and the diversity of IoT applications, resources required vary every time.

Effective Fog resource management enables IoT-enabled CPSs to concurrently execute many applications [65]. Nonetheless, the features of these applications differ amongst CPSs. For instance, in comparison the expected application service delivery time for a CPS that remotely monitors the respiratory functions of perilous patients is rather strict to a CPS which monitors the environmental statistics [97]. In addition, an application that enables a CPS do virtual reality operations manages far more data per unit time than an application that tracks vacant parking spaces. Such diverse properties play crucial roles in determining the QoS needs of applications, which could not be satisfied by Fog resource management only.

9.1 Fog Application Placement Framework

Most computing processes are performed close to data sources. This technique facilitates the use of IoT-based computing resources, but requires an efficient strategy for services placement in fog. The placement strategy makes it possible to optimize the service position in fog. The proposed FC framework consists of three layers (i.e., cloud, fog, and IoT) with N-fog colony. The conceptual framework for fog computing application orchestration involves the coordination of fog orchestration nodes, fog colonies, and the MAPE-K [12, 104] framework to enable efficient application management. This framework ensures optimal utilization of fog resources and enhances the overall performance of fog-based applications fig. 4.

In the Framework Fog Orchestration Nodes serve as controllers that manage and orchestrate the deployment, scaling, and adaptation of fog-based applications. They collect information from fog colonies and utilize the MADE-K framework for decision-making. Their role includes task allocation, load balancing, and resource provisioning within fog colonies. Fog Colonies consist of multiple fog nodes, each equipped with computational, storage, and networking resources. These colonies collectively form a distributed infrastructure for executing fog applications. Fog orchestration nodes interact with fog colonies to distribute tasks, manage resource usage, and ensure application scalability and reliability.

Fog orchestration nodes interact with fog colonies to distribute tasks and manage resources efficiently. They use insights from the MADE-K framework for informed decision-making. Fog colonies execute tasks and applications on fog nodes, ensuring timely processing and reduced latency. The MADE-K framework provides data-driven decision support to fog orchestration nodes by analysing real-time data and historical patterns. MADE-K framework, which encompasses Monitoring (M), Analysis (A), Decision-making (D), and Execution (E) with a shared knowledge base (k).

1. **Monitoring (M):** Involves the continuous collection of real-time data from fog nodes and devices. This data includes resource availability, network conditions, and application performance metrics. Monitoring ensures the framework is aware of the current state of the fog environment.
2. **Analysis (A):** Analyzes the collected data to gain insights into the performance of fog nodes, network conditions, and application requirements. This analysis helps in identifying potential bottlenecks, resource shortages, and performance issues.
3. **Decision-making (D):** Based on the analysis, the framework makes decisions regarding task allocation, resource provisioning, and load balancing. These decisions are aimed at optimizing application performance, minimizing latency, and ensuring efficient resource utilization.
4. **Execution (E):** Involves implementing the decisions made in the previous step. This includes deploying, migrating, and terminating application components on fog nodes as required.
5. **Shared Knowledge Base (K):** This is a repository of historical and real-time data, including past decisions, performance metrics, and application requirements. The knowledge base enhances decision-making accuracy by leveraging accumulated insights.

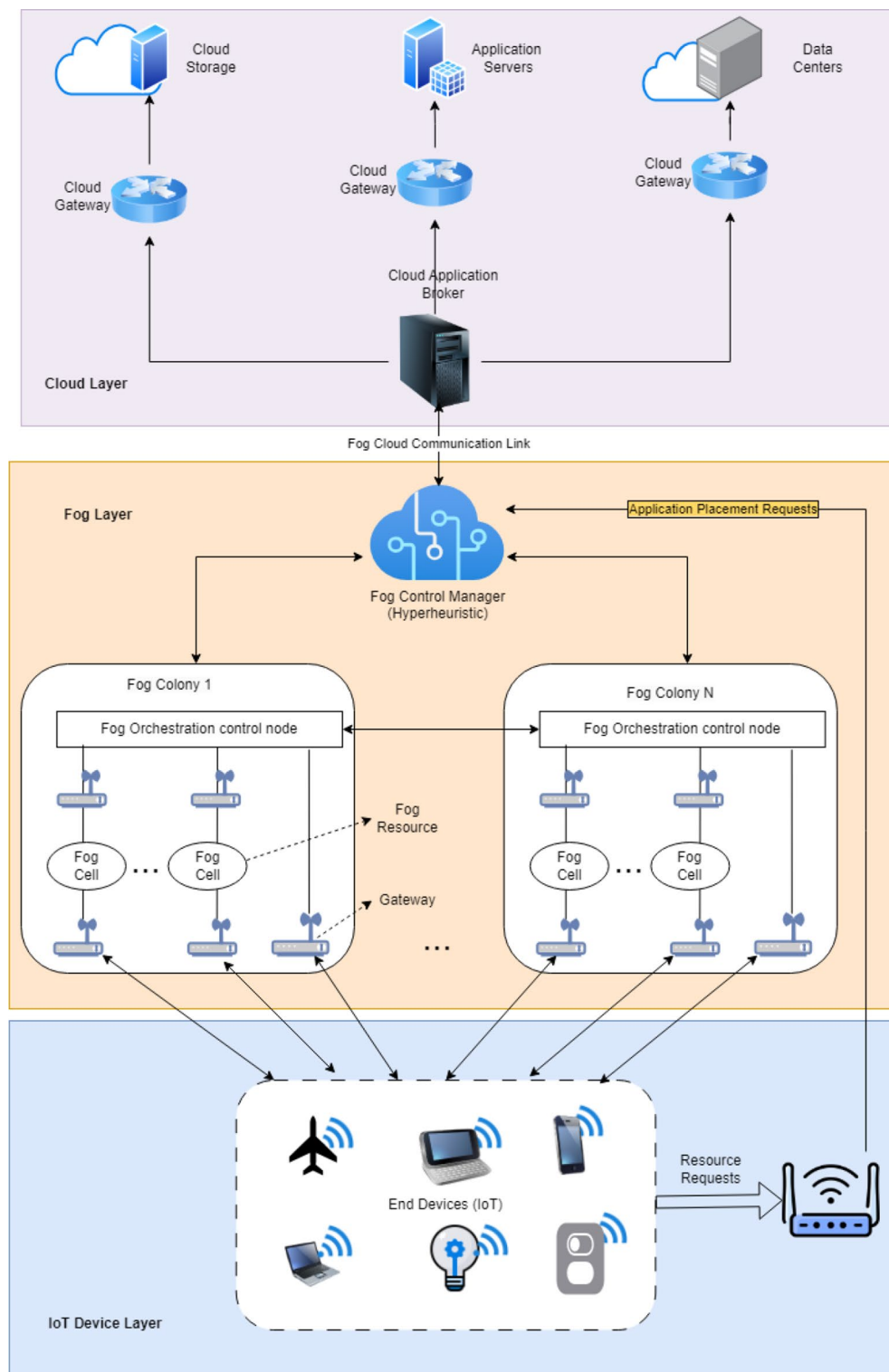


Fig. 4 Conceptual framework for Fog computing [62]

9.2 Application Placement Challenges

The difficulties faced during application placement [65] in Fog environments are detailed as follows:

- **Distributed and heterogeneous Fog nodes and Resource Limitation:** Fog nodes generally have limited processing power, storage capacity, networking capabilities, and energy capacity. They are distributed at the network's edge. Also, their communication protocols and operating systems differ. As a result, ensuring time-optimized and platform-independent application administration in Fog is challenging. In terms of resource sharing, Fog is less flexible than Cloud; for instance, Fog nodes in New Jersey cannot be used by CPSs in Sydney. These kinds of limitations restrict the execution domain of large-scale [59] IoT applications.
- **Ambiguous failures and Recovery:** Fog nodes are very susceptible to failures like power cuts and out-of-capacity issues, which impede the execution of assigned applications. Due to latency concerns, application recovery also becomes challenging.
- **Integration:** Applications operating in Fog environment often use the services provided through various computing standards. For instance, a health data analytics application which is fog based, need the Cloud-based storage solution to permit location-independent medical report. In these cases, integration of Fog infrastructure with other systems is required. But due to lack of efficient frameworks and standards prevents Fog from providing this support to applications.
- **Interoperability:** IoT applications are not interoperable because of the fundamental differences between Fog and Cloud. Existing Cloud-based IoT applications must be extensively programmed so as to take use of the advantages of FC due to a lack of compatibility.
- **Lack of a business model:** The absence of business models in fog settings hinders the budget management of users as well as profit growth of suppliers. These financial problems therefore prevent both parties from implementing Fog applications.
- **Task Distribution:** In the edge network, Fog environments work in decentralized manner. Presence of various decision-making entities upsurges the complexity of application administration in Fog settings [35], resulting in an inefficient allocation of application responsibilities among Fog nodes.
- **Heterogeneous hardware** in fog environments also present a significant challenge as fog nodes vary in computational power, storage capacity, and networking capabilities. These disparities complicate the efficient allocation of application tasks [77], as a one-size-fits-all

approach can lead to resource underutilization or overloading. Optimization algorithms can adapt to this heterogeneity by incorporating node-specific attributes, such as processing speed, bandwidth, and energy consumption, into their objective functions and constraints.

- **Latency and Energy consumption Trade-off** is also a critical consideration in application placement within fog computing, as optimizing one often comes at the cost of the other. Latency-sensitive applications require prompt responses, necessitating the use of high-performance nodes, which may lead to higher energy consumption. Conversely, prioritizing energy efficiency by allocating tasks to underutilized or energy-efficient nodes can result in increased latency, particularly for real-time applications.

10 RQ3: Fog Application Placement Problem (FAPP)

Fog Application Placement consists of mappings of each component of A to a computational node in I , satisfying all constraints and optimising a set of objective metrics O used to assess their quality. Their solution includes mappings may be many-to-many, meaning that a component may be put on one or more nodes, and a node may host several components. This SLR tries to resolve FAPP with the objective of optimising the fog - cloud system Table 3.

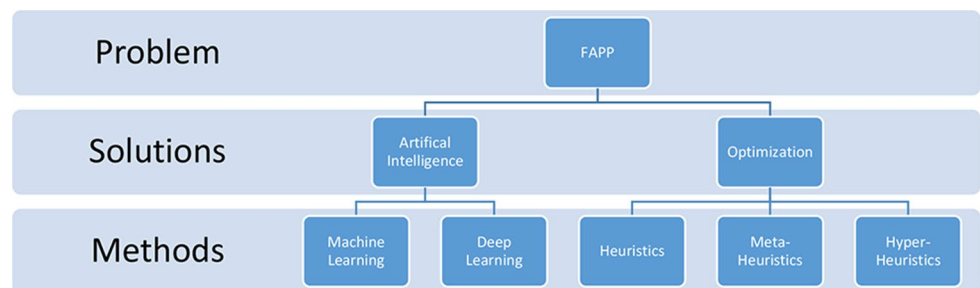
Fog application placement problem involves placing fog applications [92] in the fog layer of a fog computing architecture. The goal is to optimize the placement of fog applications in order to reduce the overall cost (e.g., energy consumption, network traffic) while satisfying various constraints (e.g., computational power, storage capacity). There are several solutions as presented in Table 1 to the fog application placement problem present in the literature but we have classified the solutions in two broad categories namely Artificial Intelligence and Optimization techniques as shown in Fig. 5. Artificial Intelligence majorly involves solutions using ML algorithms [36] e.g., neural networks, support vector machines, and decision trees to learn from data and make predictions about the best placement of fog applications. The other solution is Optimization Techniques which have been reviewed in the next section.

11 Optimization Algorithms for FAPP

To make better use of heterogeneous infrastructure, FAP algorithms must elect for each component of an application on which node of the infrastructure the given component should be placed. The placement of an application has

Table 3 Various works for the fog application placement problem

Work	Latency	Multi objective	Energy	Utilization	Execution Time	Cost	No Apps	Response time	Platform
[66]			✓	✓					Python
[52]	✓			✓					Python
[19]				✓	✓		✓	✓	-
[75]	✓		✓	✓	✓				-
[109]	✓			✓	✓				Cloud Sim
[26]				✓			✓		Python
[67]					✓		✓		Java
[69]					✓	✓		✓	iFogSim
[70]			✓						-
[34]		✓		✓	✓			✓	iFogSim
[64]	✓			✓					iFogSim2
[86]					✓				-
[61]			✓	✓		✓			-
[60]	✓				✓		✓		YAFS and iFogSim
[81]								✓	-
[78]	✓			✓					Python
[72]	✓		✓	✓		✓			-
[96].)	✓			✓				✓	Cloud Sim
[87]		✓			✓				-
[8]	✓	✓	✓			✓			-
[107]				✓		✓			-
[62]	✓	✓	✓		✓	✓		✓	MATLAB
[99]	✓		✓	✓	✓		✓		iFogSim
[2]	✓				✓				MATLAB
[101]	✓	✓	✓					✓	MapReduce
[93]	✓	✓			✓			✓	Python

Fig. 5 AI and Optimizations are the solutions to FAPP

significant impact on various metrics, such as load balancing, energy consumption, costs and resource consumption. So, the required placement should be optimized to get better results. To perform optimization there are various optimization algorithms present in literature. For our SLR, we have divided Optimization Algorithms in three broad categories as Heuristic, Meta-heuristics and hyper-heuristics as shown in Fig. 6.

12 Heuristic Approaches

Heuristic techniques seek excellent, practical solutions to optimization issues in situations when the problem's complexity or the available time does not permit accurate solution. These methods are based on experience or intuition, rather than on mathematical proofs. Heuristics are often used in situations where the problem is too complex to solve analytically or where the search space is huge to explore exhaustively. Various types of heuristics optimization methods include: genetic algorithms, simulated annealing, and Evolutionary.

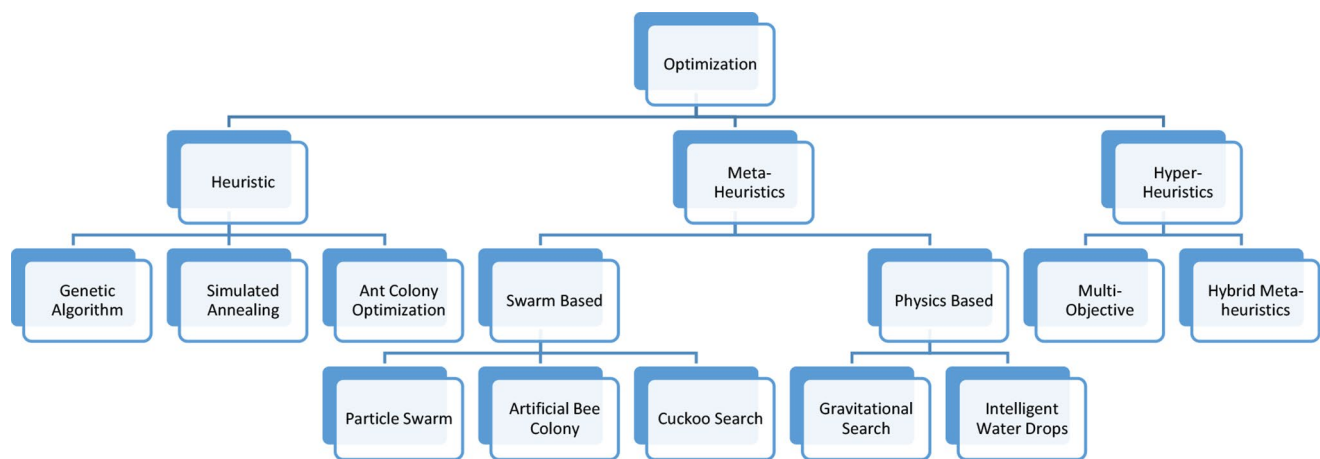


Fig. 6 Three broad categorization of Optimization techniques- Heuristics, Meta Heuristics and Hyper Heuristics

12.1 Genetic Algorithms

Genetic Algorithms (GAs) are heuristic adaptive search algorithms that comes under evolutionary algorithms and are frequently used to deliver high-quality solutions to optimization and search problems. This technique imitates the process of natural selection to find a good solution. In [23], they have used GA to solve the application placement problem. In fog computing, GA is adapted by encoding fog node configurations and application placement as chromosomes, evolving them over generations to minimize latency and maximize resource utilization. Crossover and mutation operations are tailored to respect constraints like bandwidth limitations and energy efficiency, ensuring feasible solutions. For FAPP, GA's adaptability makes it effective for dynamic task distributions in heterogeneous fog environments. GA works by maximizing $\sum_{j=1}^m U_i$ subject to constraints

$$\sum_{j=1}^m R_{ij} \leq C_i$$

where U_i is the utility of application i at fog node j , R_{ij} is the resource requirement, and

C_i is the capacity of the fog-node. GA often requires significant computational time for convergence, which may not suit real-time fog computing scenarios with dynamic workloads. GA often requires significant computational time for convergence, which may not suit real-time fog computing scenarios with dynamic workloads. Although, GA can balance latency and energy by evolving solutions to minimize both metrics, but it may prioritize energy efficiency over quick responses, especially in large-scale fog networks.

12.2 Simulated Annealing

Similar to how a thermodynamic system transitions from one energy level to another, the SA method [83] explores the set of solutions, also known as the configurational space. Thus, the creation of SA provided a mathematical connection between optimization issues and thermodynamic systems, so establishing a new basis for analysing and solving optimization problems. It is often used to resolve combinatorial optimization issues. In [18] they have applied SA for scheduling and mapping applications in fog environment. SA is modified for fog environments by incorporating a cooling schedule that considers fog node load variations and energy costs. SA explores potential placements of tasks by probabilistically accepting suboptimal solutions, helping escape local optima. By gradually focusing on energy-efficient and low-latency configurations, SA achieves near-optimal application placement under dynamic fog conditions. SA works by minimizing the objective

$$E = \sum_{j=1}^n L_i + P_i$$

where L_i is the latency and P_i is the power consumption for application i transition probability $P(\Delta E) = e^{-\Delta E/T}$ where T is the temperature. SA's slow cooling schedule can lead to prolonged optimization times, making it less efficient for handling frequent changes in fog environments. SA can explore trade-offs by probabilistically accepting suboptimal solutions, often favouring energy savings at the expense of slightly higher latency during its cooling phase.

12.3 Ant Colony Optimization

In ACO, a number of artificial ants construct solutions to the given optimization problem [32] and share data on the quality of these solutions using a communication mechanism

like that of actual ants. Various methods for optimising ant colonies have been presented. The very first ant colony optimization algorithm is called as Ant System. Inspiration for ant colony optimization comes from the foraging behaviour of actual ant colonies. This behaviour is used in artificial ant colonies to search for approximate solutions to discrete optimization issues, continuous optimization problems, and significant telecommunications challenges such as routing and load balancing. In [47] the authors have used ACO to load balance IoT tasks on the fog nodes, under response time and communication cost considerations. In outcomes, compared to the PSO-based and RR algorithms, ACO-based scheduler attains improvement in the response times of IoT applications and effectively load balances the fog nodes [15]. have also discussed ant colony in fog environment.

ACO is adapted for fog computing by representing fog nodes as graph nodes and applications as paths. Artificial ants lay pheromones to denote high-performing routes for placing tasks with minimal latency and energy usage. Constraints like SLA violations and node capacities influence pheromone evaporation rates, guiding task assignments dynamically in real-time. ACO works by maximizing

$$\sum_{i=1}^n \tau_{ij} * \eta_{ij}$$

Subject to constraints

$$C_j = \sum_{j=1}^m R_{ij}$$

where τ_{ij} is the pheromone value, η_{ij} is the heuristic information, and C_j is the capacity of fog node j . ACO can struggle with scalability as the pheromone update process becomes computationally expensive for large-scale fog networks. ACO algorithms can dynamically adjust task to placement minimizing latency, however their optimization targets especially energy-based targets hugely depends on pheromone evaporation rates, which ultimately leads to energy inefficiency especially for low-latency fog paths.

13 Meta Heuristic Approaches

Meta-heuristic is a high-level approach that guides an underlying heuristic solving a given problem. “A meta-heuristic is an iterative master process that guides [105] and modifies the operations of subordinate heuristics to efficiently produce high-quality solutions. The subordinate heuristics may be high (or low) level operations, a straightforward local search, or just a construction approach. The family of meta-heuristics consists of adaptive memory procedures, tabu search, ant systems, greedy randomised adaptive search, variable neighbourhood search, gray-wolf optimization

[101], evolutionary techniques, genetic algorithms, scatter search, neural networks, Glowworm Swarm [114], simulated annealing, and their hybrids, among others.

13.1 Swam Based Optimization

Swarm Intelligence (SI) is the emergent collective intelligence of groups of simple agents”. It refers to the collective intelligence of self-organized and decentralised systems, such as artificial groupings of simple agents (birds, insects etc.). Some examples of SI are social insect nest construction, cooperative transportation, social insect group feeding, and collective sorting and clustering. Self-organization and division of labour are two key features of SI.

13.1.1 Particle Swarm Optimization (PSO)

Motivated by social behaviour of animals e.g. bird flocking and fish schooling, PSO is a stochastic, population-based algorithm [82]. In this algorithm, the search space is probed using a population (swarm) of possible solutions (particles). Particles examine the search space according to some simple formulas. Particles are led by their individual best-known location in the search space and also the best-known position of the whole swarm (each bird utilises its own experience and others experiences). As better places are located, they will be used to direct the swarm’s movements. By repeating the process, it is hoped, but not guaranteed, that a suitable resolution will eventually be obtained. In [48], authors have introduced Frequency Particle Swarm Optimization (FPSO) for inspecting digital forensics PSO to detect and classify the Shamoon attack in fog environment. During evaluation their classifier was able to provide an accuracy (0.99998).

In fog computing, PSO is tailored by treating particles as potential solutions that represent application placements. Velocity updates consider fog-specific objectives like low response time and high resource utilization, while incorporating constraints such as energy limits. PSO’s global and local search capabilities make it suitable for real-time adaptations in dynamic environments. PSO works by minimizing $f(x) = \sum_{j=1}^n (L_i + E_i)$ with position X_i and velocity V_i updated as

$$V_i = wV_i + c_1r_1(P_{\text{best}} - X_i) + c_2r_2(G_{\text{best}} - X_i)$$

Where, w is inertia, c_1 , c_2 are coefficients, and r_1 , r_2 are random numbers. PSOs may prematurely converge to suboptimal solutions in highly dynamic and heterogeneous fog settings. Although, PSO can achieve a good balance by adjusting particle velocities based on both latency

and energy metrics, often excelling in real-time scenarios requiring low latency.

13.1.2 Artificial Bee Colony

Motivation behind ABC is the cognitive behaviour of actual honey bees in locating food sources, often called as nectar, and communicating this knowledge to rest of bees residing in hive. This method is reportedly as simple and straightforward to build as PSO. In this strategy, the artificial agents are characterised and classified as the employed bee, the observer bee, and the scout bee. Several tasks have been given to each of these bees so as to finish the operations. The hired bees concentrate on a food source and remember the location of that food source. As each hired bee is linked with a single food source, the count of employed bees equals the count of food sources. The employed bee in the hive communicates the location of the food supply to the observer bee. Next, one of the food sources is chosen for nectar collection. The scout bee is accountable for discovering new food sources and nectar. In [102], they have proposed artificial bee algorithm for collaborating fog nodes with the cloud to decrease average response time and energy cost. The outcome of the experiment tells that the suggested solution has substantial effects on decreasing the response time and energy cost.

ABC can be adapted for FAPP by encoding fog nodes and task placements as food sources. Employed and onlooker bees evaluate placement solutions based on metrics like latency and cost, while scout bees explore new configurations to avoid stagnation. Dynamic workload shifts and energy constraints guide the search, ensuring efficient resource allocation.

$$F(X) = \frac{\sum_{i=1}^n U_i}{T + E}$$

Where, U_i is utility, T is response time, and E is energy consumed. Bees update positions based on

$$X'_i = X_i + \phi (X_i - X_k)$$

Where, ϕ is a random factor and k is a neighbour. ABC can suffer from poor exploitation of promising solutions, leading to slower convergence in complex FAPP scenarios. It emphasizes energy efficiency by optimizing resource allocation, but it may fail to consistently provide low-latency responses due to slower convergence.

13.1.3 Cuckoo Search

The motivation behind Cuckoo Search Algorithm (CSA) was the behaviour of cuckoo class, e.g., brood parasites, and the features of Lévy flights, such as the flight patterns of some birds and fruit flies. CSA utilises three fundamental principles or procedures in its execution. First, each cuckoo is restricted to laying just one egg every cycle, and it chooses the nest at random. Second, the high-quality eggs and nests are passed on to the upcoming generation. The host may choose to either reject the egg or leave the nest and construct a new one. The final assumption may be approximated as a proportion, pa , of the entire n nests that are replaced with new nests using a new random solution. The technique may also be expanded to more complex situations in which each nest includes numerous eggs. In [50] The authors have used cuckoo search with Levy Walk distribution (CLW) and Flower Pollination for maintain load balance in fog as well as cloud.

CSA adapts to fog computing by representing application placements as nests and leveraging Levy flights to explore global optima under resource and SLA constraints. The algorithm discards poorly performing solutions, allowing the best-performing configurations to propagate, ensuring high scalability and efficiency in fog systems.

$$f(x) = \sum_{j=1}^n (L_i + E_i)$$

Where, L_i is latency and C_i is cost for application iii . Positions update as:

$$X_i^{t+1} = X_i^t + \alpha * \text{Levy}(\lambda)$$

Where, α is the step size and $\text{Levy}(\lambda)$ is the Levy flight distribution. CSA's reliance on Levy flights may result in inefficient exploration, particularly in environments with tightly constrained resources. CSA can focus on energy efficiency using Levy flights for global exploration but may neglect latency-critical applications in favour of conserving resources.

13.1.4 Physics Based

Physics based meta heuristics are used to proficiently crack the computationally challenging optimization problems. Researchers drew inspiration from a variety of naturally occurring physical processes to develop meta-heuristics that have produced near-optimal or ideal solutions for a number of engineering issues. There are many physics based meta heuristics present in literature [51, 80, 106] and a few are discussed as follows.

13.1.5 GSA

Physics based GSA followed the dynamics of gravity [24] and mass interaction based on the laws of gravity as well as Newton's second law of motion. Contrary to the law of gravity, which states that acceleration is inversely proportional to R^2 , according to this algorithm each particle accelerates inversely proportional to R . The authors found experimentally that use of either yields better results. In [53] to solve distributed Processing Elements scheduling and placement problem, authors used (GSA) to illustrate the noteworthy influence of fog presence on the performance of Processing Elements processing.

In fog computing, GSA is adapted by representing fog nodes and application tasks as masses in a gravitational system. Heavier masses (more optimal solutions) attract others, guiding the placement of tasks based on factors like latency, resource utilization, and energy efficiency. The gravitational constant is dynamically adjusted to account for the heterogeneity of fog environments, enabling efficient and adaptive placement strategies.

$$f(x) = \sum_{j=1}^n (L_i + P_i)$$

where L_i is latency, P_i is power consumption, and M_i is the mass of each solution. Force F_{ij} between masses i and j .

$$R_{ij} = G(t) \frac{M_i * M_j}{R_{ij} + ?}$$

where, $G(t)$ is the gravitational constant and R_{ij} is the distance between solutions. GSA's convergence heavily depends on the gravitational constant, which can be difficult to tune for the diverse demands of fog computing. GSA can effectively balances latency and energy consumption by adjusting gravitational forces, but it can lean towards energy savings at the cost of increased response times in dynamic environments.

13.1.6 IWD

IWD was inspired by water drops moving in a river as a big swarm [24]. If a fictitious water drop flows from one point of a river to the next, some significant changes will occur. The velocity and soil of water droplets will grow, whilst the soil of the riverbed will decrease. Hence, an IWD has two essential properties: its velocity and the dirt it transports. Hence, a river is composed of water drops that seek for a solution to a given issue, and the route that the water drops converge on is the answer.

IWD is tailored to fog computing by modeling the placement of tasks as water droplets navigating a terrain of fog

nodes. Each drop modifies the environment (path quality) based on resource availability, energy consumption, and latency. The algorithm's flow mechanism dynamically identifies the most efficient routes, allowing for adaptive application placement even in changing workloads.

$$f(x) = \sum_{j=1}^n (T_i + E_i)$$

where T_i is task execution time and E_i is energy consumption. Path P is updated as

$$P(t+1) = P(t) + \Delta P$$

Where, ΔP depends on the quality of the path Q and the drop's velocity V

$$V = V + \frac{\Delta P}{\eta + T_i}$$

where η is a friction factor. IWD may be computationally expensive in updating paths and handling the terrain's dynamic changes in fog environments. IWD is energy-efficient because it optimizes paths incrementally but may struggle with latency-sensitive applications due to its iterative path-update process.

5.10.3 Hyper Heuristic Optimization.

Hyper-heuristic optimization refers to a type of optimization technique that combines the principles of heuristics and metaheuristics to find good solutions to problems. Hyperheuristics are top-level strategies that select, generate, and adapt low-level heuristics to find solution of a problem [63]. A hyper-heuristic algorithm typically involves two main components: a collection of base heuristics and a selection mechanism that chooses which heuristic to implement at each step of the optimization process. The main advantage of hyper-heuristics is that they are capable to work on diverse types of problems, and can be utilized in variety of optimization problems, including combinatorial optimization, function optimization, and machine learning. Some examples of Hyper Heuristic Optimization are Hybrid Meta-heuristics, Combinatorial Evolutionary Algorithms, Adaptive Large Neighbourhood Search, Multi-Objective Hyper-Heuristics Table 4.

13.2 Hybrid Meta-Heuristics

Hybrid Meta-heuristics (HMH) is a type of Hyper-heuristic [98] optimization method that combines multiple meta-heuristics or heuristics to solve a problem. The idea behind HMH is to take advantage of the strengths of different meta-heuristics and to overcome their weaknesses by combining them in a complementary way. In HMH algorithm, different

Table 4 Evolution of Heuristics, Meta Heuristics and Hybrid Heuristic approaches

Approach	Paper	Objective	Algorithm	Performance Metrics	Key Points
Genetic Algorithm	[23]	Mapped sensors in fog nodes	Heuristics Genetic Algorithm	Convergence, Sensitivity	Minimized latency in distributing data streams across fog nodes
	[1]	Reduced latency and execution time	Heuristics GA, PCA, SVM	Latency, Execution Time, Accuracy	Increased processing speed and response time.
	[2]	Reduce Execution Time with Modified GA	Heuristics Modified GA	Execution Time	Formulated task scheduling as a permutation-based optimization problem.
	[8]	Optimally placed application modules on the underlying fog devices	Heuristics knapsack, non-dominated sorting genetic algorithm II (NSGA-II)	Latency, Total Cost, Energy Consumption	Optimally placed applications in fog devices considering applications criticality levels and security requirements
	[2]	Optimize Task Scheduling for FC	Heuristics Modified GA	Makespan and the total execution cost	Scheduling as a permutation-based optimization problem to reduce task failure rate effectively,
Simulated Annealing	[18]	Mapping of the tasks and a schedule table of their activation, and maximize QoC	Heuristics Simulated Annealing	Cost, jitter time	SA algorithm for determining the schedules and partitions, to solve optimization problem
	[17]	Maximize the QoC for the control tasks and meet the timing requirements for all tasks	Heuristics Improved Simulated Annealing	Cost, jitter time	Tasks are scheduled with a static-cyclic scheduling policy
	[54]	Load balancing to improve resource utilization in FC.	Heuristics plant growth optimization and water cycle optimization, SA	Time Delay, Cost, Energy	Hybrid load balancing approach is adopted to lower energy consumption, with experiments conducted using benchmark workflows such as GENOME and Cyber-shake.
Ant Colony Optimisation	[47]	Efficient distribution of tasks across fog nodes.	Heuristics ACO and PCO	Response Time	Two distinct scheduling algorithms are proposed for efficient load balancing of IoT tasks.
	[15]	Optimal blockchain schedule	Heuristics ACO and Block chain	Scheduling Time, Execution time	ACO is used for best assignment schedule of tasks with privacy persevering
	[108]	Complex task scheduling problem with some priority constraints	Heuristics ACO, LBPA	Energy Consumption	Laxity-Based Priority Algorithm (LBPA) Is Also Used for Obtaining Priority Sequence

Table 4 (continued)

Approach	Paper	Objective		Algorithm	Performance Metrics	Key Points
Particle Swarm Optimisation	[48]	Detection & mitigation of malware attacks	Meta Heuristics	PSO And KNN	Accuracy, Sensitivity Specificity, Precision Recall, F-Measure, confusion Matrix	PSO for inspecting digital forensics PSO to detect and classify the Shamoon attack in fog environment.
	[71]	Optimize hierarchical service process from fog to cloud	Meta Heuristics	PSO	Response Time	Game Theory. gateway of the specified domain selects the best node using PSO algorithm
	[107]	To schedule by exploiting stochasticity.	Meta Heuristics	PSO, SA	Cost, Resources	Hierarchical and distributed multi-cloud architecture
	[111]	To minimize the latency	Meta Heuristics	Multi-Objective Simplified Swarm Optimization	Convergence, Makespan, Cost	MOSSO has a faster convergence ability in fog computing task scheduling problem.
	[87]	Optimize IoT services placement with QoS	Meta Heuristics	PSO	Response Time, Service Cost, Throughput	Autonomous fog-cloud control middleware framework that maximizes the utilization of fog resources and improving QoS
	[31]	To study IoT services placement	Meta Heuristics	Modified GA	energy consumption, delay	Algorithm minimized energy consumption and applications delay violations Using Discreet PSO
	[102]	Addressed challenges arising due to nodes expansion and collaboration	Meta Heuristics	ABC	Average Time, Energy Cost	Collaborating fog nodes with the cloud has substantial effects on decreasing the response time and energy cost.
Artificial Bee Colony	[88]	Cost-effectiveness	Meta Heuristics	ABC	Runtime, Energy Consumed	ABC is used to prioritize the client request through a fitness function.
	[62]	Optimize resource scheduling	Meta Heuristics	PSO, ABC	Latency, Energy Consumption	ABC is applied to optimize the task scheduling among fog clusters based on the time and energy consumption
	[14]	To manage the workload over the edge server under	Meta Heuristics	ABC	Response Time	In ABC the objective function is set to achieve the minimum computation cost and low latency for the offloaded tasks.
	[50]	Minimized RT, PT and cost for fog	Meta Heuristics	CSO	Processing Time, Response Time, Cost	Nature inspired load balancing algorithms resulting minimum runtime.
Cuckoo Search	[33]	Allocation of the IoT services in the cloud-fog environment	Meta Heuristics	CSO, PSO	Makespan, Energy, Communication Cost	Framework allocates the best possible resources for requested IoT
	[62]	To resolve fog service placement problem	Meta Heuristics	cuckoo search	Latency, Execution, Response Time, resource utilization, cost	Framework solves problem using several interconnected colonies managed by the Multi cloud
	[16]	to optimize delay	Meta Heuristics	CSO	DELAY	Optimal cache placement that maximizes the weighted sum of STP, EE, and negative delay
	[79]	to minimize average response time and optimize resource utilization	Meta Heuristics	(MPSO) (MCSO)	resource utilization; average response time; energy consumption; and execution time	Efficient distribution of tasks across fog nodes

Table 4 (continued)

Approach	Paper	Objective		Algorithm	Performance Metrics	Key Points
Hybrid Heuristics	[113]	resource management in Smart Grid	Meta Heuristics	ACO and ABC	response time, execution time, cost	Hierarchical structure of cloud–fog computing to provide different types of computing services for SG resource management.
	[106]	Smart production lines with FC	Meta Heuristics	PSO, ACO	TIME, Energy consumption	Hybrid algorithm is used for limited computing resources, reduced energy consumption at real-time.
	[41]	Trade-off between energy consumption and task execution time.	Hyper Heuristics	multi-objective Gray-wolf optimization	Execution Time	GWO is proposed for efficient fog computing service scheduling and activation management
	[99]	to minimize energy consumption	Meta Heuristics	JAYA	energy, latency, bandwidth	A new Levy flight-based JAYA algorithm is proposed to solve the module/service placement problem.

metaheuristics or heuristics are used to explore different search space regions and to generate new results. In [79], authors suggested a bio-inspired hybrid algorithm (NBIHA) which is a fusion of modified particle swarm optimization (MPSO) and modified cat swarm optimization in fog environment. Their outcomes show that the suggested model illustrates promising results in average execution time, response time, and energy consumption, in comparison to the other scheduling techniques present in literature.

13.3 Multi-Objective Hyper-Heuristics

A multi-objective optimization problem includes various objectives, which need to be maximized or minimized depending on the problem. A MOHH algorithm is able to find a set of solutions that trade-off the different objectives, rather than finding a single optimal solution. Multi-level hyper-heuristics (MLHH) refers to the use of different levels of abstraction or different types of heuristics to solve a problem. For example, a MLHH algorithm might use a high-level heuristic to generate a set of solutions, and then use a low-level heuristic to refine these solutions. In [111], authors used an enhanced version of MOSSO (Multi-Objective Simplified Swarm Optimization) so as to resolve task scheduling problem as well as to improve the processing speed of the fog devices. Their results also illustrate that MOSSO has better convergence ability in task scheduling problem as well as proven to be a cost-effective solution for fog.

13.4 RQ5: Evaluation of Optimisation Algorithms for FAPP

The evaluation section aims to provide a comprehensive analysis of various fog application placement approaches within the context of optimization algorithms and heuristics.

This section explores the performance of different strategies across different dimensions, shedding light on their effectiveness in achieving specific objectives. Through rigorous experimentation and data collection, we assess the strengths and weaknesses of each approach to gain insights into their applicability in real-world fog computing scenarios. The Evaluation is divided into two parts first part focuses on metric-based evaluations, which evaluates Fog Utilization, SLA violation and Response time of various SOTA works collected from the literature The Next part evaluates different type of Optimization algorithms and Optimization objectives Side by Side to see which type of Algorithms are best suited for FAPP.

14 Metric Based Evaluation

This section focuses on evaluations of SOTA algorithms on the basis of metrics, we collected Fog Utilization, SLA violation and Response time of various SOTA works collected from the literature. The metrics-based evaluation presented in this paper focuses on Fog Utilization, SLA violations, and response time. The reasoning behind selection of these metrics was that these were the metrics that were present the most in the literature. These metrics comprehensively capture the core challenges of fog computing, such as efficient resource usage, maintaining service-level agreements, and minimizing latency. The selection of these metrics ensures consistency with prior studies and facilitates meaningful comparisons of state-of-the-art optimization algorithms. We also selected the most used optimization algorithms for evaluation, Including GA (Genetic Algorithm), SA (Simulated Annealing), ACO (Ant Colony Optimization), PSO (Particle Swarm Optimization), ABC (Artificial Bee Colony), CSO (Cuckoo Search Optimization), and Hybrid (a

hybrid heuristic) which combined multiple heuristic algorithms into a single one.

The evaluation in this paper is based on the findings and data reported in the reviewed studies, enabling a high-level synthesis of performance trends across various algorithms. While quantitative comparisons, such as benchmarks or results from simulations under standardized conditions, would indeed enhance the analysis, they fall outside the scope of this SLR. However, addressing this limitation is a priority for future work, where simulations and experiments will be conducted to directly compare the performance of these algorithms under consistent datasets and environments. This approach complements the current review, providing both theoretical insights and empirical validation.

14.1 Fog Utilization

Fog Utilization refers to the effective allocation and utilization of computational resources available at fog nodes for hosting and executing applications. It involves optimizing the distribution of applications across different fog nodes to achieve optimal performance, energy efficiency, and resource utilization [34, 94]. Efficient fog utilization in application placement is essential as Placing applications on nearby fog nodes minimizes data transfer distances, reducing latency and enhancing real-time processing capabilities. Also, Proper placement prevents overloading or underutilization of specific fog nodes, ensuring a balanced distribution of computing tasks. This metric indicates the proportion of computing resources (CPU, memory, storage) used in a fog node. It's calculated as:

Fog Utilization Percentage

$$= \frac{\text{Used Fog Resources}}{\text{Total Fog Resources}} \times 100$$

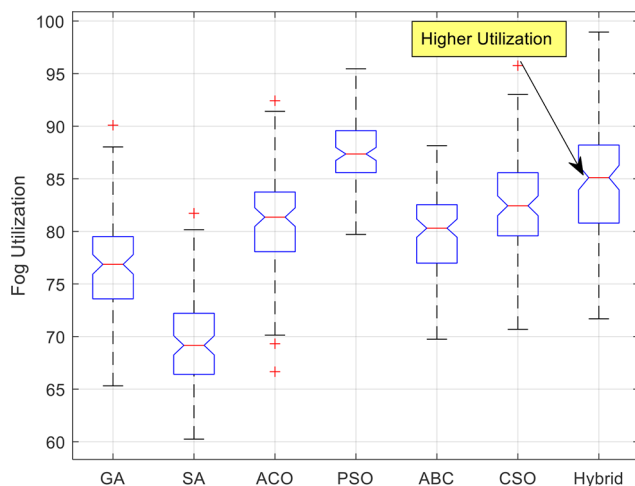


Fig. 7 Evaluation results of various algorithms in terms of Fog Utilization

The evaluation results of optimization algorithms for fog application placement, measured by the “Resource Utilization” metric are shown in Fig. 7. The figure shows Box Plot which reveals insights into the performance of each algorithm. Among the tested algorithms, PSO stands out with the highest mean resource utilization of 87.7%, while Simulated Annealing (SA) has the lowest mean of 69.8%. Notably, the “Hybrid” approach which combines various methods, yielding a mean utilization of 84.9. SA showcases the least variation (4.1 standard deviation), indicating consistent performance, followed by ACO and CSO. The hybrid approach displays the broadest range (71.6 to 98.94) in resource utilization. Considering the results, if stability and consistent resource utilization are of paramount importance, SA might be a prudent choice. On the other hand, if pushing for higher mean utilization is a priority, PSO offers an attractive option. The “Hybrid” approach might be worth considering if adaptability to diverse scenarios is required, despite the broader utilization range.

14.2 SLA Violation Rate

SLA Violation Rate refers to the frequency or proportion of instances where a service or application fails to meet the terms outlined in its Service Level Agreement (SLA) [37]. An SLA is a contractual agreement between a service provider and its customers, specifying the expected performance, availability, and quality of service. The SLA violation rate quantifies how often these agreed-upon terms are not met, indicating the extent to which service delivery falls short of expectations. SLA violation rate is a critical metric for assessing the reliability and quality of services in various contexts, including fog application placement. It's important for many reasons including Operational Efficiency, Financial Implications, and overall customer satisfaction. To Calculate the proportion of total instances where SLAs were violated over the total instances observed.

$$SLA\ Violation\ Rate = \frac{\text{Number of Violations}}{\text{Total Instances}} \times 100$$

Fog application placement strategies should aim to minimize the SLA violation rate by intelligently distributing applications across fog nodes.

The Fig. 8 depicts the evaluation of different optimization algorithms, focusing on the “SLA Violations” metric. The “Hybrid” approach achieves the lowest mean SLA violations (7.99), indicating its effectiveness in minimizing breaches of service agreements. SA, GA, ACO, and ABC exhibit similar mean values for SLA violations, ranging from 8.27 to 8.98. The “Hybrid” approach demonstrates the lowest standard deviation, suggesting more stable performance in terms of SLA compliance. CSO has the highest

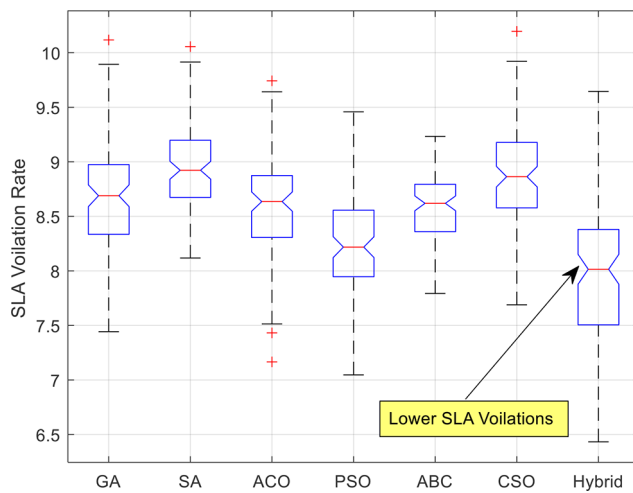


Fig. 8 Evaluation results of various algorithms in terms of SLA Violations Rates

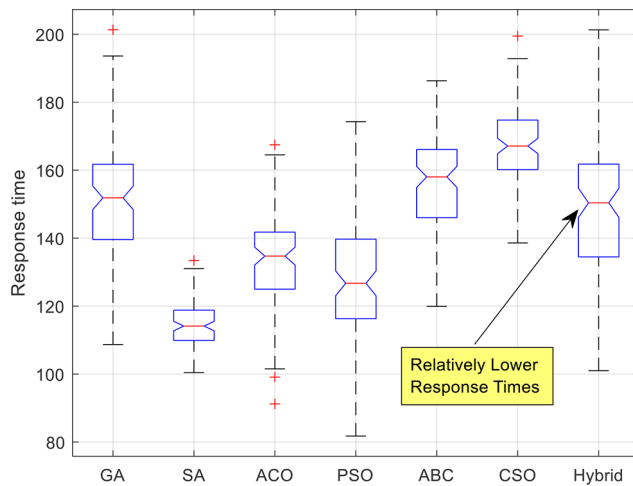


Fig. 9 Evaluation results of various algorithms in terms of Response time (ms)

mean SLA violations (8.83) and the broadest range (7.7 to 10.2). Considering the low mean and relatively low standard deviation, the “Hybrid” approach appears promising for minimizing SLA violations.

14.3 Response Time

Response Time refers to the duration it takes for a system or application to process and respond to a user’s request. In the context of fog application placement, response time is a critical performance metric that measures the speed at which applications can provide results or feedback to users [43]. It plays a vital role in determining the perceived quality of service and user experience.

Response Time

$$= \text{Time Application Placed} \\ - \text{Time Application Request Sent}$$

Response time optimization in fog application placement involves strategic placement of applications on fog nodes that can process requests quickly and efficiently. In the Evaluation Fig. 9, SA exhibits the lowest mean response time (115.1), suggesting its effectiveness in achieving quicker response times. PSO and ACO also show relatively low mean response times (128.7 and 135.15, respectively). CSO has the highest mean response time (166.3), indicating potential challenges in achieving quick responses. The “Hybrid” approach demonstrates a mean response time (149.6) comparable to other algorithms. SA seems particularly suitable for minimizing response times, making it a strong contender for applications prioritizing quick interactions. PSO and ACO also offer competitive response times, with the latter being slightly more variable. CSO may need further refinement to ensure consistently low response times. The “Hybrid” approach showcases a response time mean similar to other algorithms, but its higher standard deviation might imply a wider range of response time outcomes.

14.4 Metric Evaluation Summary

Mean references to the average fog utilization achieved by each algorithm across multiple works with the standard deviation (Std) values. A lower standard deviation indicates that the algorithm’s performance is consistent. The range (Min-Max) of values highlights the variability in fog utilization for collected works. In practical terms, selecting the most appropriate algorithm for fog application placement should align with the specific requirements of the applications. If rapid and consistent response times are paramount, SA could be a strong choice. However, if a balance between mean response time and variability is sought, PSO or ACO might be suitable. More Metrics should be considered for enhancing the performance of the system as we will see in the next section which focused on the evaluation of optimisation heuristics and Optimization Objectives. Heuristics algorithms although offer fast solutions by prioritizing one metric over the other but often fail to provide a balanced trade-off. For instance, a heuristic approach may minimize energy consumption by placing tasks on low-energy nodes, potentially causing delays due to network constraints. Meta-heuristic algorithms, such as GA and ACO, excel in balancing these objectives by exploring a wider solution space. However, these approaches can struggle with computational overhead in large-scale environments.

Hybrid heuristics, which combine features of heuristics and metaheuristics, offer a more robust approach to

managing this trade-off. For example, PSO can be hybridized with local search techniques to refine placements that initially minimize energy consumption, ensuring acceptable latency levels. Similarly, hybrid versions of SA incorporate latency penalties into their cost functions, dynamically adjusting cooling schedules to balance energy and response time. Some algorithms, like ABC and CS, are particularly energy-efficient but may fail to meet stringent latency requirements, making them more suitable for non-real-time applications. On the other hand, ACO and PSO often prioritize latency, making them ideal for real-time systems, though at the expense of slightly higher energy use.

15 Evaluation of Heuristics and Objectives

The evaluation section aims to provide a comprehensive analysis of various fog application placement approaches within the context of optimization algorithms and heuristics. This section explores the performance of different strategies across different dimensions, shedding light on their effectiveness in achieving specific objectives. Through rigorous experimentation and data collection, we assess the strengths and weaknesses of each approach to gain insights into their applicability in real-world fog computing scenarios. The evaluation is structured into distinct categories that reflect the diversity of fog application placement strategies:

15.1 Single Heuristic Single Objective (SHSO)

In Fog Application Placement, a “single heuristic single objective” scenario refers to using a single rule or approach to place applications on fog nodes in a way that optimizes a single objective. This could be minimizing latency, maximizing throughput, or minimizing energy consumption. Suppose a fleet of IoT devices that collect sensor data and need real-time analysis. Your objective is to minimize latency. A single heuristic, such as “nearest-fog” placement, could be used to place each application on the fog node that’s physically closest to the IoT device generating the data. The placement decision is solely based on minimizing the communication delay.

15.2 Single Heuristic Multiple Objective (SHMO)

In this scenario, a single heuristic is used to guide the placement of applications on fog nodes considering multiple conflicting objectives. This could involve minimizing latency, maximizing resource utilization, and minimizing energy consumption simultaneously. Consider a scenario where you want to deploy applications that process both real-time sensor data and perform periodic data analytics. A single

heuristic, like the “weighted score” approach, might consider factors such as proximity to data sources (low latency), available resources (high utilization), and energy efficiency. The heuristic combines these factors to make placement decisions that balance multiple objectives.

15.3 Hybrid Heuristic Single Objective (HHSO)

A hybrid heuristic in fog application placement involves combining multiple heuristic strategies to optimize a single objective, such as minimizing latency or maximizing resource utilization. Let’s say you’re tasked with placing applications in a fog network where some applications are latency-sensitive and others are resource-intensive. You could use a hybrid approach that combines a “latency-based heuristic” with a “resource utilization heuristic.” The former would prioritize placing latency-sensitive apps closer to data sources, while the latter would optimize resource utilization by considering CPU and memory availability on fog nodes.

15.4 Hybrid Heuristic Multiple Objective (HHMO)

In this scenario, a hybrid heuristic combines multiple heuristic strategies to optimize multiple objectives simultaneously in fog application placement. When deploying applications in a fog network for a smart city project, objectives include minimizing latency, maximizing resource utilization, and reducing overall energy consumption. A hybrid heuristic could combine an “energy-aware placement heuristic” with a “latency and resource trade-off heuristic.” This approach would consider both energy efficiency and latency/resource trade-offs when placing applications on fog nodes.

The evaluation results shown in Fig. 10 depict the average performance of various types heuristics and Optimization objectives in terms of Fog Utilisation. The variations in the evaluation results for different fog application placement approaches can be attributed to several factors that influence the performance of each approach. The SHSO achieved lowest fog utilization 78.7%. As this approach uses a single rule or strategy to optimize a single objective, resulting in the lower outcome. As a result, the optimization might not be well-rounded. SHMO on the other hand attained the average fog utilization of 82.5%. Combining multiple heuristics can bring diverse strategies to the table. This approach might effectively tackle the single objective by leveraging the strengths of each heuristic and mitigating their individual weaknesses. Thus, Hybrid Heuristics approaches demonstrated a higher performance compared to Single Heuristics in the evaluation. Whereas the HHMO Yielded the highest fog utilization for fog utilization during application placement.

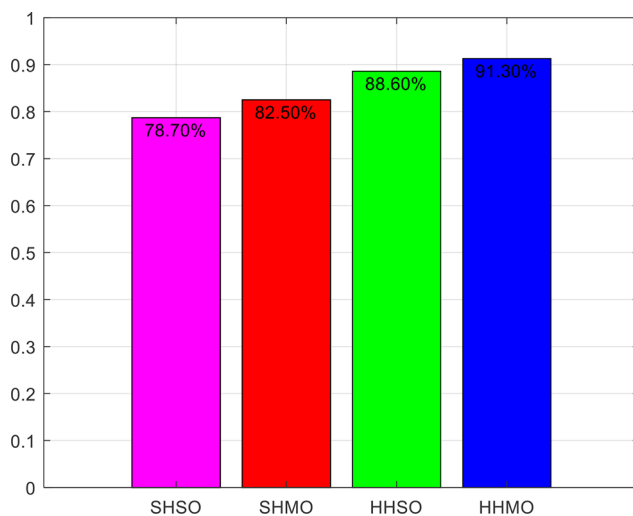


Fig. 10 Evaluation of Heuristics (Single and Hybrid) and Objectives (Single & Multiple)

The results indicate the effectiveness of different fog application placement strategies based on the combination of heuristics and the optimization objectives. The HHMO approach stands out as the most successful in achieving the desired outcomes, suggesting its potential superiority in balancing and optimizing multiple conflicting objectives in fog computing scenarios.

15.5 Limitations of Optimization Algorithms

For FAPP, GAs can intelligently explore the search space for optimal task placement by combining and mutating potential solutions, ensuring scalability and adaptability in dynamic fog environments. However, GA need significant computational time for convergence, which may not suit real-time fog computing scenarios with dynamic workloads. In FAPP, SA is well-suited for minimizing latency and SLA violations by iteratively improving placement strategies while avoiding local optima. SAs suffer with a slow cooling schedule which leads to prolonged optimization times, making it less efficient for handling frequent changes in fog environments.

Whereas, ACO uses pheromone trails to guide the search for efficient routes. It can determine optimal data routing and task allocation paths, balancing load and reducing response times in a distributed fog network. But ACOs usually struggle with scalability as the pheromone update process becomes computationally expensive for large-scale fog networks. Another swarm method, PSO can efficiently allocate resources by quickly adapting to changing demands and identifying high-performance configurations. However, PSO can prematurely converge to suboptimal solutions in highly dynamic and heterogeneous fog settings. Mimicking the foraging behaviour of honeybees, ABC uses

a collaborative search mechanism to optimize solutions. However, ABC algorithms suffer from poor exploitation of promising solutions, leading to slower convergence in complex FAPP scenarios. CS on the end can discover innovative placement strategies by identifying high-performing nodes while discarding inefficient configurations, However, CSs reliance on Levy flights may result in inefficient exploration, particularly in environments with tightly constrained resources.

Heuristic approaches tackle FAPP by providing quick, rule-based solutions tailored to specific problems, offering simplicity and efficiency for static or less complex fog environments. Unlike metaheuristic approaches, they lack the adaptability to explore diverse and dynamic solution spaces. Metaheuristic approaches address FAPP by searching for near-optimal solutions in vast and dynamic problem spaces, accommodating real-time fog computing challenges. Although, they surpass heuristic methods by adapting to changing scenarios but may require more computational effort. In order to tackle FAPP, hyper-heuristic optimization takes a variety of low-level heuristics and designs versatile and adaptive solutions for a range of fog scenarios. Physics based approaches are based on natural principles, whereas hyper heuristics are revolved around exploiting heuristic diversity for adaptability.

Therefore, Hybrid meta-heuristics can solve FAPP by integrating advantages of several optimization approaches to attain a better exploration and exploitation balance for Fog environments which are complex. Integrating rather than selecting algorithms, they build on hyper heuristic strategies and improve synergy and robustness. In contrast, Multi-objective hyper heuristics treat FAPP through the simultaneous optimisation of conflicting objectives (e.g., minimisation of latency and maximisation of resource utilisation) across multiple heterogeneous fog settings. Instead, they extend to hybrid approaches by explicitly targeting multiple goals, and bridging the trade-offs through tailored and balanced solutions.

16 RQ6: Discussions and Future Works

With the increasing reliance on distributed systems, the optimization of application placement has become increasingly important in order to maximize resource utilization, minimize latencies, and reduce costs. This review evaluated various academic papers concerning fog computing application placement, specifically concentrating on Optimization algorithms. The examination of these papers addressed six research questions and revealed outcomes that illuminated the problems the authors tackled. Moreover, this analysis facilitated the recognition of unexplored hurdles demanding

further research and innovative proposals within the scope of this investigation.

There are a number of promising directions for future research in this area. One direction is the development of more advanced optimization algorithms that can better handle the complexity and scale of modern distributed systems. This could involve the development of algorithms that can handle large-scale optimization problems with multiple objectives, such as minimizing latencies while maximizing resource utilization. Another direction is the integration of optimization algorithms with machine learning techniques, such as reinforcement learning, to enable more dynamic and adaptable application placement strategies. This could involve the use of machine learning algorithms to continuously learn and adapt to changing workloads and resource availability in order to optimize application placement in real-time.

From the evaluations done in this paper, it is clear that focusing solely on a single objective, potentially neglecting other important factors does reduce the performance. While a single heuristic guides the optimization, the presence of multiple objectives can lead to trade-offs. Balancing conflicting objectives could make the optimization process more complex. The complexity of optimizing multiple objectives often benefits from a hybrid approach [5]. Combining heuristics could enable a more comprehensive exploration of the solution space, leading to better trade-offs and solutions. Also, the choice of optimization algorithms in the hybrid approaches can significantly impact results. Algorithms that complement each other's strengths can lead to better overall performance. The quality and appropriateness of the selected heuristics in each approach play a critical role. Well-designed heuristics are more likely to lead to effective optimization. The relative importance of objectives (e.g., resource utilization, SLA compliance) can also influence how different approaches prioritize them. Approaches can benefit from more comprehensive exploration of the solution space, potentially discovering better solutions in multi-objective scenarios.

For complex optimization tasks, selecting the appropriate hybrid meta-heuristic results in the highest performance as seen from comparison of single vs. hybrid heuristics [46], but this does not always mean that combining more sophisticated algorithms will provide the best outcomes. As goal is to create an algorithm that examines the benefits of each scheme and offers a minimal performance advantage over algorithms that operate independently [7]. To compute answers with much reduced time and space complexity, hyper heuristic approaches can establish an automated search space from which generalised solutions may be filtered out, hyper heuristic approaches combine multiple-heuristics to construct a solution. Thus, employing

hyper-heuristic algorithms can offer significant advantages when it comes to optimizing application placement in fog computing environments.

SLA violations and response time are critical metrics in fog computing, often presenting trade-offs that optimization algorithms must carefully balance. Penalties ensue when algorithm try to optimize response time at the expense of SLA violation, since minimizing SLA violations often requires spreading tasks around and the nodes which have sufficient resources could slow down response time. Conversely, prioritizing low response times usually leads to resource contention and SLA breaches for most applications. This trade-off is something that optimization algorithms can address utilizing multi-objectives and considering both metrics at the same time. For instance, GA [56] can be modified to employ a weighted fitness functions to evolve solutions to minimize SLA violations and response time. Nevertheless, their frequent reliance on intensive iterations can result in slow decisions, therefore reducing real time applicability.

To maintain the adherence of SLA and minimize response time, hybrid heuristics working together by combining SA with local search techniques fine tune solutions. Nevertheless, due to their slow convergence, they are not suitable for workloads with high variability. Other algorithms, such as ABC and CS, have also been effective at minimizing SLA violations but are likely to underperform under latency sensitive applications, as they are exploration focused. However, there are limitations on these approaches that affect their ability to dynamically prioritize metrics according to workload shifts or demands of an application. Approaches such as blockchain also have been explored recently [20] to tackle SLA violations. Future work can explore adaptive hybrid approaches which can adjust objectives based on real-time node conditions, resource demands, enabling a more effective trade-off between latency, SLA adherence and energy consumption in diverse fog environments. This will enable better trade-offs, ensuring both high performance and reliability in fog environments.

The FAPP can also benefit by integrating ML and DL [6, 36] approaches and Reinforcement learning (RL) can allow for a dynamic task offloading by learning in real time optimal placement strategies in fog environments based on the changing workloads and resource conditions [55]. More recently, development of Deep neural networks (DNNs) can model complex relationships between system parameters to predict the optimal system configuration to achieve minimal SLA violation and lowest response time with optimizing energy usages. Finally, Utilization of the hybrid approaches combining ML/DL with standard metaheuristics could further improve the adaptability and scalability of solutions [11].

With an increasing number of IoT devices relying on fog computing for data management and processing, energy efficient algorithms for the low power IoT environment are important. To that end, newer algorithms must consider the constrained computational and energy resources of IoT devices, while providing reliable application placement and task execution. Optimization of IoT nodes for energy efficiency can maximize operational lifespan of IoT nodes, reduce overall system energy consumption and environmental impact utilizing newer optimization algorithms, such as Hyper-heuristics.

Author Contributions Dear Sir, Contributions of authors for submitted paper titled “Evaluation of Optimization Algorithm for Application Placement Problem in Fog Computing: A Systematic Review” are as follows Ankur Goswami (A): Conceptualization, Literature Search, Data Extraction, Data Analysis, and Writing—Original Draft Preparation. Ankur led the systematic review process, including developing research questions, defining inclusion and exclusion criteria, and conducting the comprehensive literature search. He was responsible for extracting, analysing, and synthesizing data on optimization algorithms for the application placement problem in fog computing, and for drafting the final manuscript. Kirit Modi (B): Supervision, Methodology, Validation, and Writing—Review & Editing. Kirit supervised the overall research process, contributed to refining the SLR methodology, and provided guidance on data validation. He also reviewed and edited the manuscript, ensuring coherence and alignment with research goals. His expertise helped ensure that the research questions and analysis met the requirements for rigorous SLR standards. Chirag Patel (C): Data Curation, Investigation, Visualization, and Writing—Review & Editing. Chirag contributed to data curation and provided additional insights during the investigation phase, particularly focusing on categorizing optimization algorithms and metrics for fog computing applications. He assisted in visualizing the findings, contributing figures and tables, and collaborated on manuscript revisions to enhance clarity and presentation of the systematic review findings.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing Interests The authors declare no competing interests.

References

- 1 Abdellatif AAH, Singh A, Aldribi A, Ortega-Mansilla A, Ibrahim M (2022) A novel framework for fog-assisted smart healthcare system with workload optimization. *Comput Intell Neurosci* 2022. <https://doi.org/10.1155/2022/4174805>
- 2 Abohamama AS, El-Ghamry A, Hamouda E (2022) Real-time task scheduling algorithm for IoT-Based applications in the cloud–fog environment. *J Netw Syst Manage* 30(4):54. <https://doi.org/10.1007/s10922-022-09664-6>
- 3 AlAhmad AS, Kahtan H, Alzoubi YI, Ali O, Jaradat A (2021) Mobile cloud computing models security issues: a systematic review. *J Netw Comput Appl* 190:103152. <https://doi.org/10.1016/j.jnca.2021.103152>
- 4 Alatoun K, Matrouk K, Mohammed MA, Nedoma J, Martinek R, Zmij P (2022) A novel low-latency and energy-efficient task scheduling framework for internet of medical things in an edge fog cloud system. *Sensors* 22(14) Article 14. <https://doi.org/10.3390/s22145327>
- 5 Ali A, Adeel Ali Shah S, Shloul A, Assam T, Ghadi MY, Lim Y, S., Zia A (2024) Multiobjective Harris Hawks optimization-based task scheduling in cloud-fog computing. *IEEE Internet Things J* 11: (13), 24334–24352. <https://doi.org/10.1109/JIOT.2024.3391024>
- 6 Alsadie D (2024a) A comprehensive review of AI techniques for resource management in fog computing: trends, challenges, and future directions *IEEE Access* 12:118007–118059. <https://doi.org/10.1109/ACCESS.2024.3447097>
- 7 Alsadie D (2024b) Advancements in heuristic task scheduling for IoT applications in fog-cloud computing: challenges and prospects. *PeerJ Comput Sci* 10:e2128. <https://doi.org/10.7717/peerj-cs.2128>
- 8 Al-Tarawneh MAB (2022) Bi-objective optimization of application placement in fog computing environments. *J Ambient Intell Humaniz Comput* 13(1):445–468. <https://doi.org/10.1007/s12652-021-02910-w>
- 9 Alwakeel AM (2021) An overview of fog computing and edge computing security and privacy issues. *Sensors* 21(24). <https://doi.org/10.3390/s21248226>
- 10 Apat HK, Nayak R, Sahoo B (2023) A comprehensive review on internet of things application placement in fog computing environment. *Internet Things*, 100866
- 11 Aqib M, Kumar D, Tripathi S (2023) Machine learning for fog computing: Review, opportunities and a fog application classifier and scheduler. *Wireless Pers Commun* 129(2):853–880. <https://doi.org/10.1007/s11277-022-10160-y>
- 12 Arcaini P, Riccobene E, Scandurra P (2015) Modeling and analyzing MAPE-K feedback loops for self-adaptation. In: *ACM 10th international symposium on software engineering for adaptive and self-managing systems*. IEEE, pp. 13–23. <https://ieeexplore.ieee.org/abstract/document/7194653/>
- 13 Asghari A, Sohrabi MK (2024) Server placement in mobile cloud computing: a comprehensive survey for edge computing, fog computing and cloudlet. *Comput Sci Rev* 51:100616. <https://doi.org/10.1016/j.cosrev.2023.100616>
- 14 Babar M, Khan MS, Din A, Ali F, Habib U, Kwak KS (2021) Intelligent computation offloading for IoT applications in scalable edge computing using artificial bee colony optimization. *Complexity* 2021. <https://doi.org/10.1155/2021/5563531>
- 15 Baniata H, Anaqreh A, Kertesz A (2021) PF-BTS: a privacy-aware fog-enhanced blockchain-assisted task scheduling. *Inf Process Manage* 58(1). <https://doi.org/10.1016/j.ipm.2020.102393>
- 16 Bani-Bakr A, Hindia MN, Dimiyati K, Hanafi E, Tengku Mohmed Noor Izam TF (2021) Multi-objective caching optimization for wireless backhauled fog radio access network. *Symmetry* 13(4):Article4. <https://doi.org/10.3390/sym13040708>
- 17 Barzegaran M, Cervin A, Pop P (2019) Towards quality-of-control-aware scheduling of industrial applications on fog computing platforms. In *Proceedings of the workshop on fog computing and the IoT*, Association for Computing Machinery, New York, pp. 1–5. <https://doi.org/10.1145/3313150.3313217>
- 18 Barzegaran M, Cervin A, Pop P (2020) Performance optimization of control applications on fog computing platforms using scheduling and isolation. *IEEE Access* 8:104085–104098. <https://doi.org/10.1109/ACCESS.2020.2999322>
- 19 Barzegaran M, Pop P (2022) Extensibility-aware fog computing platform configuration for mixed-criticality applications. *J Syst Architect* 133. <https://doi.org/10.1016/j.sysarc.2022.102776>
- 20 Battula SK, Garg S, Naha R, Amin MB, Kang B, Aghasian E (2022) A blockchain-based framework for automatic SLA management in fog computing environments. *J Supercomputing*

- 78(15):16647–16677. <https://doi.org/10.1007/s11227-022-04545-w>
- 21 Bonomi F, Milito R, Zhu J, Addepalli S (2012) Fog computing and its role in the internet of things. In: MCC'12 - Proceedings of the 1st ACM mobile cloud computing workshop, Association for Computing Machinery, New York, pp. 13–15. <https://doi.org/10.1145/2342509.2342513>
 - 22 Brogi A, Forti S, Guerrero C, Lera I (2020) How to place your apps in the fog: state of the art and open challenges. *Softw - Pract Experience* 50(5):719–740. <https://doi.org/10.1002/spe.2766>
 - 23 Canali C, Lancellotti R (2019) GASP: genetic algorithms for service placement in fog computing systems. *Algorithms* 12(10). <https://doi.org/10.3390/a12100201>
 - 24 Chattopadhyay S, Marik A, Pramanik R (2022) A brief overview of physics-inspired metaheuristic optimization techniques. <http://arxiv.org/abs/2201.12810>
 - 25 Chen M, Hao Y, Li Y, Lai C, Wu D (2015) On the computation offloading at ad hoc cloudlet: architecture and service models. *IEEE Commun Magazine* 53(6):18–24
 - 26 Choi J, Ahn S (2019) Scalable service placement in the fog computing environment for the IoT-based smart city. *J Inform Process Syst* 15(2):440–448. <https://doi.org/10.3745/JIPS.03.0113>
 - 27 Costa B, Bachiega J, Carvalho LR, Araujo APF (2023) Orchestration in fog computing: a comprehensive survey. *ACM-CSUR* 55(2). <https://doi.org/10.1145/3486221>
 - 28 Hegde DNSBA, Abhijit S, C. S., Ambesange S (2024) FedCure: A heterogeneity-aware personalized federated learning framework for intelligent healthcare applications in IoMT environments. *IEEE Access* 12 15867–15883. <https://doi.org/10.1109/ACCESS.2024.3357514>
 - 29 Darbandi M (2017) Proposing new intelligence algorithm for suggesting better services to cloud users based on Kalman Filtering. *J Comput Sci Appl*
 - 30 Darbandi M, Haghighi S, Hajiali M, Khabir A (2018) Prediction and estimation of next demands of cloud users based on their comments in CRM and previous usages. In: 2018 Int Conf Communication Comput Internet Things (IC3IoT), IEEE, pp. 81–86. <https://doi.org/10.1109/IC3IoT.2018.8668119>
 - 31 Djemai T, Stolf P, Monteil T, Pierson J-M (2020) Mobility support for energy and QoS aware IoT services placement in the fog. In: 2020 international conference on software, telecommunications and computer networks (SoftCOM), IEEE, pp. 1–7. <https://doi.org/10.23919/SoftCOM50211.2020.9238236>
 - 32 Dorigo M, Birattari M, Stutzle T (2006) Ant colony optimization. *IEEE Comput Intell Mag* 1(4):28–39. <https://doi.org/10.1109/MCI.2006.329691>
 - 33 Dubey K, Sharma SC, Kumar M (2022) A secure IoT applications allocation framework for integrated fog-cloud environment. *J Grid Comput* 20(1):5. <https://doi.org/10.1007/s10723-021-09591-x>
 - 34 Eslami M, Sakhaei-nia M (2021) A novel service deployment policy in fog computing considering the degree of availability and fog landscape utilization using multiobjective evolutionary algorithms. In: 2021 12th international conference on information and knowledge technology (IKT), IEEE, pp. 163–168. <https://doi.org/10.1109/IKT54664.2021.9685175>
 - 35 Etemadi M, Ghoobaei-Arani M, Shahidinejad A (2020) Resource provisioning for IoT services in the fog computing environment: an autonomic approach. *Comput Commun* 161:109–131
 - 36 Fahimullah M, Ahvar S, Agarwal M, Trocan M (2024) Machine learning-based solutions for resource management in fog computing. *Multimedia Tools Appl* 83(8):23019–23045. <https://doi.org/10.1007/s11042-023-16399-2>
 - 37 Faraji-Mehmandar M, Jabbehdari S, Haj S, Javadi H (2021) A proactive fog service provisioning framework for internet of things applications: an autonomic approach. *Trans Emerg Telecommunications Technol*, 32(11), e4342
 - 38 Gasmi K, Dilek S, Tosun S, Ozdemir S (2022) A survey on computation offloading and service placement in fog computing-based IoT. *J Supercomputing* 78(2):1983–2014. <https://doi.org/10.1007/s11227-021-03941-y>
 - 39 Goudarzi M, Palaniswami M, Buyya R (2021) A distributed application placement and migration management techniques for edge and fog computing environments. In: Proc 16th conf comput sci intell syst. pp. 37–56. <https://doi.org/10.15439/2021F005>
 - 40 Laghari AA, Jumani AK, Laghari RA (2021) Review and state of art of fog computing. *Archives of Computational Methods in Engineering* 28:3631–3643. <https://doi.org/10.1007/s11831-020-09517-y>
 - 41 Hashemi SM, Sahafi A, Rahmani AM, Bohloul M (2022) GWO-SA: gray wolf optimization algorithm for service activation management in fog computing. *IEEE Access* 10:107846–107863. <https://doi.org/10.1109/ACCESS.2022.3212439>
 - 42 Heidari A, Shishehlou H, Darbandi M, Navimipour NJ, Yalcin S (2024) A reliable method for data aggregation on the industrial internet of things using a hybrid optimization algorithm and density correlation degree. *Cluster Comput* 27(6):7521–7539. <https://doi.org/10.1007/s10586-024-04351-4>
 - 43 Herrera JL, Galán-Jiménez J, Berrocal J, Murillo JM (2021) Optimizing the response time in sdn-fog environments for time-critical applications. *IEEE Internet Things J* 8(23):17172–17185
 - 44 Hossain MR, Whaiduzzaman M, Barros A, Tuly SR, Mahi MJN, Roy S, Fidge C, Buyya R (2021) A scheduling-based dynamic fog computing framework for augmenting resource utilization. *Simul Model Pract Theory* 111:102336. <https://doi.org/10.1016/j.simpat.2021.102336>
 - 45 Hu P, Dhelim S, Ning H, Qiu T (2017) Survey on fog computing: Architecture, key technologies, applications and open issues. *J Netw Comput Appl* 98(April):27–42. <https://doi.org/10.1016/j.jnca.2017.09.002>
 - 46 Hussain SM, Begh GR (2022) Hybrid heuristic algorithm for cost-efficient QoS aware task scheduling in fog-cloud environment. *J Comput Sci* 64:101828. <https://doi.org/10.1016/j.jocs.2022.101828>
 - 47 Hussein MK, Mousa MH (2020) Efficient task offloading for IoT-based applications in fog computing using ant colony optimization. *IEEE Access* 8:37191–37201. <https://doi.org/10.1109/ACCESS.2020.2975741>
 - 48 Hwaitat AKA, Amin M, Almomani O, Al-Zahrani M, Asaifi MRM, Althunibat RKK, A., Alsaaidah A (2020) Improved security particle swarm optimization (PSO) algorithm to detect radio jamming attacks in mobile networks. *Int J Adv Comput Sci Appl* 11(4). <https://doi.org/10.14569/IJACSA.2020.0110480>
 - 49 Islam MM, Ramezani F, Lu HY, Naderpour M (2023) Optimal placement of applications in the fog environment: a systematic literature review. *J Parallel Distrib Comput* 174:46–69. <https://doi.org/10.1016/j.jpdc.2022.12.001>
 - 50 Javaid N, Butt AA, Latif K, Rehman A (2019) Cloud and fog based integrated environment for load balancing using Cuckoo Levy distribution and flower pollination for smart homes. In: 2019 international conference on computer and information sciences (ICCIS), IEEE, pp. 1–6. <https://doi.org/10.1109/ICCISci.2019.8716467>
 - 51 Kabirzadeh S, Rahbari D, Nickray M (2017) A hyper heuristic algorithm for scheduling of fog networks. In: 2017 21st conf open innovations association (FRUCT), IEEE, pp. 148–155. <https://doi.org/10.23919/FRUCT.2017.8250177>
 - 52 Karagiannis V, Frangoudis PA, Dustdar S, Schulte S (2023) Context-aware routing in fog computing systems. *IEEE Trans Cloud Comput* 11(1):532–549. <https://doi.org/10.1109/TCC.2021.3102996>

- 53 Karamoozian AH, Aboulhamid EM (2019) On the fog-cloud cooperation: how fog computing can address latency concerns of IoT applications. In: 2019 Fourth int conf fog mob edge comput (FMEC), IEEE, pp. 166–172. <https://doi.org/10.1109/FMEC.2019.8795320>
- 54 Kaur M, Aron R (2022) An energy-efficient load balancing approach for scientific workflows in fog computing. *Wireless Pers Commun* 125(4):3549–3573. <https://doi.org/10.1007/s11277-022-09724-9>
- 55 Keshavarz Haddadha P, Rezvani MH, MollaMotalebi M, Shankar A (2024) Machine learning methods for service placement: a systematic review. *Artif Intell Rev* 57(3):61. <https://doi.org/10.1007/s10462-023-10684-0>
- 56 Kochovski P, Pašćinski U, Stankovski V, Ciglaric M (2022) Pareto-optimised fog storage services with novel service-level agreement specification. *Appl Sci* 12(7) Article 7. <https://doi.org/10.3390/app12073308>
- 57 Kumari A, Tanwar S, Tyagi S, Kumar N, Obaidat MS, Rodrigues JJPC (2019) Fog computing for smart grid systems in the 5G environment: challenges and solutions. *IEEE Wirel Commun* 26(3):47–53. <https://doi.org/10.1109/MWC.2019.1800356>
- 58 Kunal S, Saha A, Amin R (2019) An overview of cloud-fog computing: architectures, applications with security challenges. *Secur Priv*, 2(4), e72
- 59 Lee G, Saad W, Bennis M (2017) An online optimization framework for distributed fog network formation with minimal latency. <http://arxiv.org/abs/1710.05239>
- 60 Lera I, Guerrero C, Juiz C (2019) YAFS: a simulator for IoT scenarios in fog computing. *IEEE Access* 7:91745–91758. <https://doi.org/10.1109/ACCESS.2019.2927895>
- 61 Lim J (2022) Latency-aware task scheduling for IoT applications based on artificial intelligence with partitioning in small-scale fog computing environments. *Sensors* 22(19):7326. <https://doi.org/10.3390/s22197326>
- 62 Liu C, Wang J, Zhou L, Rezaeipana A (2022) Solving the multi-objective problem of IoT service placement in fog computing using cuckoo search algorithm. *Neural Process Lett* 54(3):1823–1854. <https://doi.org/10.1007/s11063-021-10708-2>
- 63 Maashi M, Kendall G, Özcan E (2015) Choice function based hyper-heuristics for multi-objective optimization. *Appl Soft Comput* 28:312–326. <https://doi.org/10.1016/j.asoc.2014.12.012>
- 64 Mahmud R, Pallewatta S, Goudarzi M, Buyya R (2021) IFog-Sim2: an extended iFogSim simulator for mobility, clustering, and microservice management in edge and fog computing environments (arXiv:2109.05636). arXiv. <http://arxiv.org/abs/2109.05636>
- 65 Mahmud R, Ramamohanarao K, Buyya R (2020) Application management in fog computing environments: a taxonomy, review and future directions. *ACM-CSUR* 53(4). <https://doi.org/10.1145/3403955>
- 66 Mahmud R, Toosi AN (2021) Con-Pi: a distributed container-based edge and fog computing framework. <https://doi.org/10.1109/JIOT.2021.3103053>
- 67 Mann ZA (2022) Decentralized application placement in fog computing. *IEEE Trans Parallel Distrib Syst* 33(12):3262–3273. <https://doi.org/10.1109/TPDS.2022.3148985>
- 68 Martin BA, Michaud F, Banks D, Mosenia F, Zolfonoon R, Irwan S, Schrecker S, Zao JK (2017) OpenFog security requirements and approaches. In: 2017 IEEE Fog world congress (FWC), IEEE, pp. 1–6. <https://doi.org/10.1109/FWC.2017.8368537>
- 69 Martin JP (2020) Orchestration mechanisms for enabling distributed processing in the fog computing environment
- 70 Masri A, Al-Jabi M (2021) Toward IoT fog computing-enabled system energy consumption modeling and optimization by adaptive TCP/IP protocol. *PeerJ Comput Sci* 7:e653. <https://doi.org/10.7717/peerj-cs.653>
- 71 Mirzapour-Moshizi M, Sattari-Naeini V (2022) QoE aware application placement in Fog environment using SAW game theory method [Preprint]. In Review. <https://doi.org/10.21203/rs.3.rs-2133563/v1>
- 72 Mondragón-Ruiz G, Tenorio-Trigoso A, Castillo-Cara M, Caminero B, Carrión C (2021) An experimental study of fog and cloud computing in CEP-based real-time IoT applications. *J Cloud Comput* 10(1):32. <https://doi.org/10.1186/s13677-021-00245-7>
- 73 Mouradian C, Naboulsi D, Yangui S, Glitho RH, Morrow MJ, Polakos PA (2018) A comprehensive survey on fog computing: state-of-the-art and research challenges. *IEEE Commun Surv Tutor* 20(1):416–464. <https://doi.org/10.1109/COMST.2017.2771153>
- 74 Mukherjee M, Shu L, Wang D (2018) Survey of fog computing: fundamental, network applications, and research challenges. *IEEE Commun Surv Tutor* 20(3):1826–1857. <https://doi.org/10.1109/COMST.2018.2814571>
- 75 Naha RK, Garg S (2019) Multi-criteria-based dynamic user behaviour aware resource allocation in fog computing (arXiv:1912.08319). arXiv. <http://arxiv.org/abs/1912.08319>
- 76 Nayeri ZM, Ghafarian T, Javadi B (2021) Application placement in fog computing with AI approach: taxonomy and a state of the art survey. *J Netw Comput Appl* 185(January):103078. <https://doi.org/10.1016/j.jnca.2021.103078>
- 77 Pallewatta S, Kostakos V, Buyya R (2019) Microservices-based IoT application placement within heterogeneous and resource constrained fog computing environments. In: *Proc 12th IEEE/ACM int conf util cloud comput, Association for Computing Machinery New York*, pp. 71–81. <https://doi.org/10.1145/3344341.3368800>
- 78 Qian J (2021) A decomposed deep training solution for fog computing platforms
- 79 Rafique H, Shah MA, Islam SU, Maqsood T, Khan S, Maple C (2019) A novel bio-inspired hybrid algorithm (NBIHA) for efficient resource management in fog computing. *IEEE Access* 7:115760–115773. <https://doi.org/10.1109/ACCESS.2019.2924958>
- 80 Rahbari D, Kabirzadeh S, Nickray M (2018) A security aware scheduling in fog computing by hyper heuristic algorithm. In: *Proc – 3rd Iran conf signal process intell syst (ICSPIS 2017)*, IEEE, pp. 87–92. <https://doi.org/10.1109/ICSPIS.2017.8311595>
- 81 Rathod D, Chowdhary G (2020) Service priority queuing model-based internet of things middleware for load balancing among fog computing centres. *Int J Auton Comput* 3(3/4):219. <https://doi.org/10.1504/IJAC.2020.114376>
- 82 Rezaee Jordehi A, Jasni J (2011) Heuristic methods for solution of FACTS optimization problem in power systems. In: *Proc – 2011 IEEE student conf res dev*, IEEE, pp. 30–35. <https://doi.org/10.1109/SCORED.2011.6148703>
- 83 Rutenbar RA (1989) Simulated annealing algorithms: an overview. *IEEE Circuits Devices Mag* 5(1):19–26
- 84 Sabireen H, Neelanarayanan V (2021) A review on fog computing: Architecture, fog with IoT, algorithms and research challenges. *ICT Express* 7(2):162–176. <https://doi.org/10.1016/j.ict.2021.05.004>
- 85 Salaht FA, Desprez F, Lebre A (2020) An overview of service placement problem in fog and edge computing. *ACM-CSUR* 53(3). <https://doi.org/10.1145/3391196>
- 86 Salem A (2022) Mathematical models for task assignment with service level agreement in fog computing networks [Master of Applied Science, Carleton University]. <https://doi.org/10.22215/etd/2022-15117>
- 87 Salimian M, Ghobaei-Arani M, Shahidinejad A (2022) An evolutionary multi-objective optimization technique to deploy the IoT services in fog-enabled networks: An autonomous approach.

- Appl Artif Intell 36(1):2008149. <https://doi.org/10.1080/08839514.2021.2008149>
- 88 Sharma S, Saini H (2019) A novel four-tier architecture for delay aware scheduling and load balancing in fog environment. *Sus. Com. Inf Syst* 24:100355. <https://doi.org/10.1016/j.suscom.2019.100355>
 - 89 Shi Y, Ding G, Wang H, E Roman H, Lu S (2015) The fog computing service for healthcare. In: 2015 2nd int symp future inform communication technol ubiquitous healthc (Ubi-HealthTech), IEEE, pp. 1–5. <https://ieeexplore.ieee.org/abstract/document/7203325>
 - 90 Singh Chhillar R (2022) 6739 RUCHIKA. *J Theoretical Appl Inform Technol* 100:22
 - 91 Singh SP, Nayyar A, Kumar R, Sharma A (2019) Fog computing: from architecture to edge computing and big data processing. *J Supercomputing* 75(4):2070–2105. <https://doi.org/10.1007/s11227-018-2701-2>
 - 92 Smolka S, Mann ZÁ (2022) Evaluation of fog application placement algorithms: a survey. *Computing* 104(6):1397–1423. <https://doi.org/10.1007/s00607-021-01031-8>
 - 93 Talavera F, Lera I, Juiz C, Guerrero C (2024) Optimizing fog colony layout and service placement through genetic algorithms and hierarchical clustering. *Expert Syst Appl* 254:124372. <https://doi.org/10.1016/j.eswa.2024.124372>
 - 94 Tang S (2023) Performance modeling and optimization for a fog-based IoT platform. *IoT* 4(2):183–201. <https://doi.org/10.3390/iot4020010>
 - 95 Torabi E, Ghobaei-Arani M, Shahidinejad A (2024) A learning-based data and task placement mechanism for IoT applications in fog computing: a context-aware approach. *J Supercomputing* 80(15):21726–21763. <https://doi.org/10.1007/s11227-024-06278-4>
 - 96 Tsega A (2020) Designing a deadline aware data offloading algorithm for fog computing environment
 - 97 Tuli S, Mahmud R, Tuli S, Buyya R (2018) FogBus: a blockchain-based lightweight framework for edge and fog computing. <http://arxiv.org/abs/1811.11978>
 - 98 Urli T (2015) Hybrid meta-heuristics for combinatorial optimization. *Constraints* 20(4):473–473. <https://doi.org/10.1007/s10601-015-9209-7>
 - 99 Vadde U, Kompalli VS (2022) Energy efficient service placement in fog computing. *PeerJ Comput Sci* 8:e1035. <https://doi.org/10.7717/peerj-cs.1035>
 - 100 Vahid Dastjerdi A, Gupta H, Calheiros RN, Ghosh SK, Buyya R (2016) Fog computing: principles, architectures, and applications
 - 101 Vakili A, Al-Khafaji HMR, Darbandi M, Heidari A, Jafari Navimipour N, Unal M (2024) A new service composition method in the cloud-based internet of things environment using a grey wolf optimization algorithm and MapReduce framework. *Concurr Comp: Pract Exp* 36(16):e8091. <https://doi.org/10.1002/cpe.8091>
 - 102 Vakilian S, Moravvej SV, Fanian A (2021) Using the artificial bee colony (ABC) algorithm in collaboration with the fog nodes in the internet of things three-layer architecture. In: 29th Iran conf electr eng (ICEE 2021), IEEE, pp. 509–513. <https://doi.org/10.1109/ICEE52715.2021.9544399>
 - 103 Vaquero LM, Roderio-Merino L, Caceres J, Lindner M (2008) A break in the clouds: towards a cloud definition. *ACM Sigcomm Comput Communication Rev* 39(1):50–55
 - 104 Vizcarrondo J, Aguilar J, Exposito E, Subias A (2017) MAPE-K as a service-oriented architecture. *IEEE Lat Am Trans* 15(6):1163–1175
 - 105 Voß S (2001) Meta-heuristics: The state of the art. In: Lecture notes in artificial intelligence (subseries of lecture notes in computer science (Vol. 2148, pp. 1–23). https://doi.org/10.1007/3-540-45612-0_1
 - 106 Wang J, Li D (2019) Task scheduling based on a hybrid heuristic algorithm for smart production line with fog computing. *Sens (Switzerland)* 19(5). <https://doi.org/10.3390/s19051023>
 - 107 Wei W, Li H, Yang W (2023) Cost-effective stochastic resource placement in edge clouds with horizontal and vertical sharing. *Future Gener Comput Syst* 138:213–225. <https://doi.org/10.1016/j.future.2022.08.016>
 - 108 Xu J, Hao Z, Zhang R, Sun X (2019) A method based on the combination of laxity and ant colony system for cloud-fog task scheduling. *IEEE Access* 7:116218–116226. <https://doi.org/10.1109/ACCESS.2019.2936116>
 - 109 Yakubu IZ, Muhammed L, Musa ZA, Matinja ZI, Adamu IM (2021) A multi agent based dynamic resource allocation in fog-cloud computing environment. *Trends Sci* 18(22):413. <https://doi.org/10.48048/tis.2021.413>
 - 110 Yassa S, Chelouah R, Kadima H, Granado B (2013) Multi-objective approach for energy-aware workflow scheduling in cloud computing environments. *Sci World J*. <https://doi.org/10.1155/2013/350934>
 - 111 Yeh WC, Lai CM, Tseng KC (2019) Fog computing task scheduling optimization based on multi-objective simplified swarm optimization. *J Phys Conf Ser* 1411(1). <https://doi.org/10.1088/1742-6596/1411/1/012007>
 - 112 Yousefpour A, Fung C, Nguyen T, Kadiyala K, Jalali F, Niakanlahiji A, Kong J, Jue JP (2019) All one needs to know about fog computing and related edge computing paradigms: a complete survey. *J Syst Architect* 98:289–330. <https://doi.org/10.1016/j.sysarc.2019.02.009>
 - 113 Zahoor S, Javaid S, Javaid N, Ashraf M, Ishmanov F, Afzal MK (2018) Cloud–fog–based smart grid model for efficient resource management. *Sustainability* 10(6) Article 6. <https://doi.org/10.3390/su10062079>
 - 114 Zambouri K, Darbandi M, Nassr M, Heidari A, Navimipour NJ, Yalcin S (2024) A GSO-based multi-objective technique for performance optimization of blockchain-based industrial internet of things. *Int J Commun Syst* 37(15):e5886. <https://doi.org/10.1002/dac.5886>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.